

Voxel-based block variational quantum linear solver: a hybrid quantum–classical method for static analysis of solids

Feng Wu, Chen Li, Li Zhu, Yuxiang Yang, and Xu Guo*

*State Key Laboratory of Structural Analysis, Optimization and CAE Software for Industrial Equipment,
School of Mechanics and Aerospace Engineering, Dalian University of Technology, Dalian 116024, Liaoning, P. R. China*

In solid mechanics, finite element discretization of large-scale static problems produces large sparse linear systems whose solution requires substantial computation time and memory. The variational quantum linear solver (VQLS) offers a hybrid quantum–classical route, but its use in quantum finite element analysis is limited by the decomposition of nonunitary matrices, barren plateaus, and the measurement cost of expectation values. We propose a voxel-based block variational quantum linear solver (Voxel-BVQLS) that combines structured matrix decomposition, the principle of minimum potential energy, and batched quantum tests. First, we construct an LCU decomposition of the stiffness matrix from the recursive block-banded structure of voxel-grid finite element matrices, with the number of unitary terms bounded independently of the problem size. Second, we optimize the ansatz parameters using a minimum-potential-energy objective in place of a conventional VQLS loss function, thereby mitigating barren plateaus in the studied problems. Third, we introduce a block-Hadamard test whose circuit directly estimates weighted sums of multiple inner products, reducing the number of circuit configurations required per iteration. We assessed the proposed method in noiseless classical simulations using three examples. These examples show that the method reduces both the number of unitary terms in the LCU decomposition and the number of iterations required to converge, while still yielding solutions of finite accuracy. Voxel-BVQLS thus provides a structured hybrid quantum–classical framework for quantum finite element analysis on regular grids.

Keywords: variational quantum linear solver, quantum finite element method, voxel grid, linear combination of unitaries, block-Hadamard test

1. Introduction

Advances in computing power have continually driven numerical analysis in solid mechanics [1, 2]. The finite element method (FEM) has evolved alongside classical computers and has become one of the most important numerical methods in solid mechanics [3–5]. FEM partitions the problem domain into elements, constructs element stiffness equations, and assembles a global equilibrium system. Fine discretization of very large structures often produces large sparse linear systems [6, 7]. As the number of degrees of freedom grows to millions or beyond, solution time, memory use, and parallel communication overhead typically increase substantially [8, 9]. At billions of degrees of freedom, large-scale parallel computing resources are usually required, which imposes a significant computational bottleneck. New computing approaches that alleviate the cost of classical finite element solvers are therefore of considerable interest.

Quantum computing offers a different way to process information in scientific computation [10, 11]. Superposition, entanglement, and interference allow quantum algorithms to manipulate information in high-dimensional state spaces. These features may offer computational advantages for particular tasks [12, 13]. Applications in quantum chemistry and quantum simulation

*) Corresponding author: Xu Guo. E-mail: guoxu@dlut.edu.cn

have shown promise [14–16]. At the intersection of scientific computing and mechanics, quantum algorithms have been studied for partial differential equations [17], materials and structural analysis [18], and fluid mechanics [19–21]. Solid mechanics, however, often involves heterogeneous materials, complex geometries, and varied boundary conditions [22, 23]. The entries and sparsity patterns of the resulting stiffness matrices depend on the particular problem, complicating uniform quantum encodings and circuit implementations. Moreover, most governing equations in solid mechanics, especially static equilibrium equations, do not directly take the form of a unitary Schrödinger evolution suitable for quantum simulation [24, 25]. Quantum computing for solid mechanics therefore remains at an early stage. Recent studies have constructed quantum Hamiltonians for solid dynamics through energy-conservation mappings [26], or developed decomposition-free variational solvers for structural statics [27]. Most results nevertheless concern equation mappings or small-scale proof-of-principle demonstrations. End-to-end quantum speedup for general solid mechanics problems has not been established. Methods tailored to the structure of solid mechanics problems are therefore needed.

The main computational tasks in FEM include solving large sparse linear systems and eigenvalue problems. Both are closely related to quantum linear algebra. Embedding quantum subroutines in finite element workflows is therefore a route for research at the interface of quantum computing and solid mechanics. Here, the quantum finite element method denotes a hybrid quantum–classical framework. Geometry modeling, mesh discretization, element calculations, and global assembly remain classical, while a quantum or hybrid algorithm solves the discrete algebraic system or estimates selected physical quantities [28]. In the gate model, two representative routes for finite element linear systems are quantum linear systems algorithms and the variational quantum linear solver (VQLS). The HHL algorithm is an example of the former. It uses quantum phase estimation and controlled rotations to prepare a solution state proportional to $\mathbf{K}^{-1}\mathbf{f}$ [29]. Under ideal assumptions, HHL-type algorithms may offer exponential speedup. However, a finite element complexity analysis found that the advantage over classical algorithms is generally at most polynomial [30]. HHL-type methods are thus better suited to estimating a small number of solution-state observables.

VQLS casts a linear system as a hybrid quantum–classical optimization problem [31, 32]. Its core idea is to prepare a normalized trial solution state with the ansatz circuit $\mathbf{V}(\boldsymbol{\alpha})$ and to iteratively update its parameters with a classical optimizer, so that the state approaches the exact solution. VQLS can use shallower circuits than phase-estimation-based quantum linear systems algorithms, making it a leading approach in recent quantum finite element studies. Improvements have focused on objective functions, matrix encodings, and ansatz design [33–36]. Variational quantum methods have already been applied to several finite element and related partial differential equation problems. Trahan et al. [37] used Pauli decomposition to study discrete finite element systems for the one-dimensional steady Poisson, transient heat conduction, and wave equations. Ali and Kabel [38] studied variational solutions of the Poisson equation on a noiseless simulator and a quantum device. They reported substantial limitations on current hardware. Arora et al. [39] examined a quantum finite element workflow for steady heat conduction with different elements, material properties, and boundary conditions. Liu et al. [40] combined FEM with a variational quantum eigensolver to analyze the natural vibrations of trusses, beams, and continua. Although these studies demonstrate the potential of VQLS for shallow circuits and problem-aware design, VQLS still faces three interrelated challenges in solid mechanics. First, trainability of parameterized circuits is not guaranteed. Certain random ansatz circuits and global objective functions can exhibit barren plateaus [41, 42]. Second, a quantum representation of the stiffness matrix may contain many unitary terms. An n -qubit system has 4^n Pauli strings. Without exploiting matrix structure, the nonzero terms and their controlled implementations may incur substantial circuit and measurement costs [43–45]. A common global VQLS loss contains $\mathbf{K}^T\mathbf{K}$, so representing $\mathbf{K}$ by M terms in an LCU decomposition leads to $O(M^2)$ inner products in the direct expansion, each of which also requires enough measurements to control statistical error [32, 37].

To address these challenges for real symmetric positive-definite finite element stiffness systems, we propose the voxel-based block variational quantum linear solver (Voxel-BVQLS). Voxel-BVQLS exploits regular voxel structure to reduce both the number of LCU unitary terms and the circuit configurations needed to evaluate the minimum-potential-energy objective and its gradient. The main contributions are as follows.

- (1) We develop an LCU decomposition for stiffness matrices on voxel grids. It exploits the recursive block-banded structure of the finite element matrix together with cyclic permutation matrices and Euler’s formula, and its number of unitary terms is bounded independently of the problem dimension.
- (2) We construct a variational objective from the principle of minimum potential energy. For a constrained symmetric positive-definite stiffness matrix $\mathbf{K}$, the quadratic form associated with linear-static total potential energy has the unique minimizer $\mathbf{K}^{-1}\mathbf{f}$ over the full displacement space, and the resulting objective can mitigate barren plateaus in the studied settings.
- (3) We propose a block-Hadamard test and use it to construct the block variational solver. Quantum multiplexors and a

block-partitioning strategy allow one circuit to estimate weighted sums of multiple inner products, which reduces the number of circuit configurations required per iteration.

(4) We test the method in noiseless classical simulations on a two-dimensional truss, a plane-stress model, and a three-dimensional steady-state heat-conduction model with one degree of freedom per node. All three examples produce approximate displacement or temperature fields, and the structured LCU uses fewer unitary terms than the Pauli decomposition.

The remainder of this paper is organized as follows. Section 2 reviews quantum computing fundamentals, LCU decomposition, VQLS, and its main challenges in solid mechanics. Section 3 presents the voxel-grid stiffness-matrix LCU decomposition and the minimum-potential-energy formulation. Section 4 develops the block-Hadamard test and Voxel-BVQLS. Section 5 reports three numerical examples. Section 6 summarizes the conclusions and limitations.

2. Quantum finite element method and VQLS

Quantum finite element analysis combines FEM with quantum computing. Its basic approach retains established classical steps for geometry modeling, mesh discretization, element calculations, and global assembly. The computationally demanding algebraic solve is then expressed in a form suitable for a quantum computer. After discretization and imposition of the required boundary conditions, the finite element equilibrium equation is

$$\mathbf{K}\mathbf{u} = \mathbf{f}, \quad (1)$$

where $\mathbf{K} \in \mathbb{R}^{N \times N}$ is the finite element system matrix after boundary conditions have been imposed. The vectors $\mathbf{u}, \mathbf{f} \in \mathbb{R}^N$ are the unknown displacement and load vectors, respectively. Here, N is the number of unknown degrees of freedom.

In the quantum finite element workflow used here, the finite element equations are assembled classically and solved by a hybrid quantum–classical algorithm. On the quantum side, the VQLS encodes the right-hand-side vector as a quantum state, represents the system matrix as an LCU, and prepares a trial solution with a parameterized quantum circuit $\mathbf{V}(\boldsymbol{\alpha})$. Quantum measurements estimate the objective function. On the classical side, an optimizer updates $\boldsymbol{\alpha}$ from the measurement results. The two sides alternate until a stopping criterion is met.

This section is organized as follows: Section 2.1 introduces quantum amplitude encoding and basic gates, Section 2.2 presents matrix LCU decomposition, Section 2.3 reviews the VQLS framework including the ansatz, loss functions, and the Hadamard test, and Section 2.4 discusses the main challenges of applying VQLS to solid mechanics and outlines the proposed solutions.

2.1 Quantum amplitude encoding and basic quantum gates

To solve Eq. (1) with VQLS, the right-hand-side vector must first be encoded as a quantum state. The basic information unit of a quantum computer is the qubit. A qubit has two computational basis states, $|0\rangle$ and $|1\rangle$, corresponding to classical bit values 0 and 1. Unlike a classical bit, a qubit may occupy a linear combination of these basis states: $|\mathbf{a}\rangle = a_0|0\rangle + a_1|1\rangle$. The complex numbers a_0 and a_1 are probability amplitudes. Measurement yields $|0\rangle$ and $|1\rangle$ with probabilities $|a_0|^2$ and $|a_1|^2$, respectively. Hence, $|a_0|^2 + |a_1|^2 = 1$.

An n -qubit system has 2^n computational basis states, and a general state can be expanded as $|\mathbf{a}\rangle = \sum_{i=0}^{2^n-1} a_i|i\rangle$. Here $|i\rangle$ denotes the n -bit basis state corresponding to integer i . A quantum state corresponds to a normalized complex vector of length 2^n . Conversely, any nonzero vector $\mathbf{x} \in \mathbb{C}^{2^n}$ can be normalized and written as

$$|\mathbf{x}\rangle = \frac{1}{\|\mathbf{x}\|_2} \sum_{i=0}^{2^n-1} x_i|i\rangle. \quad (2)$$

Equation (2) defines amplitude encoding. The normalized quantum states of the displacement and load vectors in Eq. (1) are

$$\begin{aligned} |\mathbf{f}\rangle &= \frac{1}{\|\mathbf{f}\|_2} \sum_{i=0}^{2^n-1} f_i|i\rangle, \quad \mathbf{f} = (f_0, f_1, \dots, f_{N-1})^T \\ |\mathbf{u}\rangle &= \frac{1}{\|\mathbf{u}\|_2} \sum_{i=0}^{2^n-1} u_i|i\rangle, \quad \mathbf{u} = (u_0, u_1, \dots, u_{N-1})^T. \end{aligned} \quad (3)$$

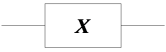
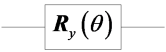
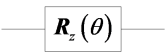
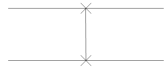
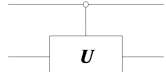
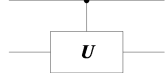
The computational basis state $|i\rangle$ is labeled by the n -bit binary representation of integer i .

Classical logic gates change bit values, whereas quantum gates change quantum states in the gate model. A deterministic quantum gate acting on a closed system is represented by a unitary matrix. Unitarity preserves the norm of the state. A quantum register usually starts in the readily prepared state $|0\rangle^{\otimes n} = |0\rangle|0\rangle \cdots |0\rangle$. For any target pure state, a unitary matrix $\mathbf{F}$ exists such that

$$|\mathbf{f}\rangle = \mathbf{F}|0\rangle^{\otimes n}, \quad (4)$$

where $\mathbf{F}$ is an $N \times N$ unitary matrix. Equation (4) shows that the load vector can be represented as a quantum state. Any unitary transformation can be built from a universal set of elementary gates. Table 1 lists the single-qubit and two-qubit gates used below, along with their circuit symbols.

Table 1 Single-qubit and two-qubit gates used in this work

	Basic gate	Unitary matrix	Quantum circuit
Single-qubit gates	Hadamard gate	$\mathbf{H} = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$	
	Pauli-X gate	$\mathbf{X} = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$	
	R_y rotation gate	$\mathbf{R}_y(\theta) = \begin{bmatrix} \cos \theta/2 & -\sin \theta/2 \\ \sin \theta/2 & \cos \theta/2 \end{bmatrix}$	
	R_z rotation gate	$\mathbf{R}_z(\theta) = \begin{bmatrix} e^{-i\theta/2} & 0 \\ 0 & e^{i\theta/2} \end{bmatrix}$	
Two-qubit gates	SWAP gate	$\begin{bmatrix} 1 & & & \\ & 1 & & \\ & & 1 & \\ & & & 1 \end{bmatrix}$	
	Open-controlled U gate	$\begin{bmatrix} \mathbf{U} & \\ & \mathbf{I}_{2 \times 2} \end{bmatrix}$	
	Controlled- U gate	$\begin{bmatrix} \mathbf{I}_{2 \times 2} & \\ & \mathbf{U} \end{bmatrix}$	

The ansatz circuits also use the CNOT gate, which is the controlled- U gate with $\mathbf{U} = \mathbf{X}$. All subsequent circuits are constructed from the gates listed in Table 1.

2.2 LCU decomposition

In the gate model, deterministic coherent evolution is implemented by unitary operators. The constrained finite element stiffness matrix $\mathbf{K}$ is real symmetric positive definite but generally does not satisfy $\mathbf{K}^H \mathbf{K} = \mathbf{I}$. It therefore cannot act directly as a quantum gate. To represent $\mathbf{K}$ in a quantum circuit, we write it as a weighted sum of unitary matrices:

$$\mathbf{K} = \sum_{i=0}^{M-1} a_i \mathbf{K}_i, \quad (5)$$

where $\mathbf{K}_i$ is unitary and $a_i \in \mathbb{C}$ is its coefficient. Equation (5) expresses a nonunitary matrix as a linear combination of unitaries. We call this an LCU decomposition of the stiffness matrix. Each $a_i \mathbf{K}_i$ is a unitary term, and M is the number of terms. A common LCU construction uses Pauli decomposition. The single-qubit Pauli matrices are

$$\boldsymbol{\sigma}_0 = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \quad \boldsymbol{\sigma}_1 = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}, \quad \boldsymbol{\sigma}_2 = \begin{bmatrix} 0 & -i \\ i & 0 \end{bmatrix}, \quad \boldsymbol{\sigma}_3 = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}. \quad (6)$$

Thus $\boldsymbol{\sigma}_0 = \mathbf{I}$, $\boldsymbol{\sigma}_1 = \mathbf{X}$, $\boldsymbol{\sigma}_2 = \mathbf{Y}$, and $\boldsymbol{\sigma}_3 = \mathbf{Z}$. The i -th n -qubit Pauli string is

$$\mathbf{K}_i = \boldsymbol{\sigma}_{i_0} \otimes \boldsymbol{\sigma}_{i_1} \otimes \cdots \otimes \boldsymbol{\sigma}_{i_{n-1}}. \quad (7)$$

Each position in the tensor product corresponds to one qubit. To enumerate the Pauli strings with a single integer, write i in base four with n digits:

$$i = \sum_{t=0}^{n-1} i_t \times 4^{n-1-t} = i_0 \times 4^{n-1} + i_1 \times 4^{n-2} + \cdots + i_{n-1} \times 4^0, \quad (8)$$

where $i_t \in \{0, 1, 2, 3\}$. There are $4^n = N^2$ candidate Pauli strings, which satisfy the trace-orthogonality relation $\text{tr}(\mathbf{K}_i \mathbf{K}_j) = 2^n \delta_{ij}$. The coefficients in the full expansion are therefore

$$a_i = \frac{1}{2^n} \text{tr}(\mathbf{K} \mathbf{K}_i). \quad (9)$$

Equation (9) gives the coefficients by projecting the stiffness matrix onto the Pauli strings. Several efficient Pauli-decomposition algorithms have been proposed, including tensorized Pauli decomposition (TPD) [50] and MSPD [51]. MSPD exploits matrix sparsity and uses a fast Hadamard transform to accelerate the decomposition. It is among the most efficient reported approaches.

2.3 Overview of VQLS

Building on quantum amplitude encoding and LCU decomposition, we review the VQLS framework. It has three components: an ansatz circuit that prepares a trial solution, a loss function that measures solution quality, and a quantum measurement procedure that estimates the loss. The following subsections introduce each component.

2.3.1 Ansatz

We first describe the ansatz used to approximate the unknown displacement vector in Eq. (1). Equation (3) defines the target displacement state $|\mathbf{u}\rangle$. VQLS uses a parameterized unitary $\mathbf{V}(\boldsymbol{\alpha})$ to prepare a normalized trial state

$$|\mathbf{u}(\boldsymbol{\alpha})\rangle = \mathbf{V}(\boldsymbol{\alpha}) |0\rangle^{\otimes n} = \mathbf{V}(\boldsymbol{\alpha}) \mathbf{z}_n. \quad (10)$$

This state approximates $|\mathbf{u}\rangle$. The circuit implementing $\mathbf{V}(\boldsymbol{\alpha})$ is the ansatz circuit, also called a parameterized quantum circuit (PQC). We denote the vector corresponding to $|0\rangle^{\otimes n}$ by $\mathbf{z}_n = (1, 0, \dots, 0)^T \in \mathbb{R}^N$. Figure 1 shows a representative ansatz with N_α parameters. Alternating $\mathbf{R}_y$ and CNOT gates act on $|0\rangle^{\otimes 6}$ and encode a displacement vector of dimension 2^6 .

The $\mathbf{R}_y$ gates in Fig. 1 adjust the computational-basis amplitudes, while the CNOT gates create correlations between qubits. Additional rotation and entangling layers can increase ansatz expressivity. They also increase parameter count, circuit depth, and optimization difficulty. Ansatz design must therefore balance expressivity against trainability.

The state $|\mathbf{u}(\boldsymbol{\alpha})\rangle$ is normalized and specifies only the direction of the displacement vector. A classical scalar c sets its magnitude. From Eq. (10), the displacement vector is

$$\mathbf{u} = c \mathbf{V}(\boldsymbol{\alpha}) \mathbf{z}_n, \quad c = \|\mathbf{u}\|_2. \quad (11)$$

Substituting Eq. (11) into Eq. (1) gives

$$\mathbf{K} c \mathbf{V}(\boldsymbol{\alpha}) \mathbf{z}_n = \mathbf{f}. \quad (12)$$

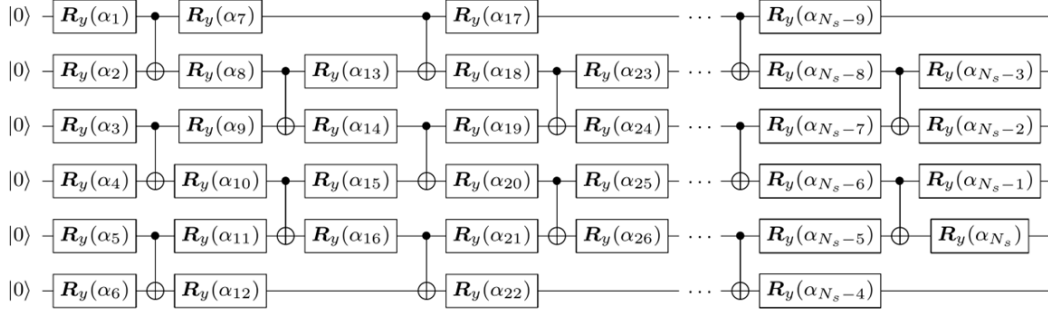


Figure 1 Real-amplitude ansatz circuit with alternating R_y and CNOT gates

Its quantum-state form is

$$c\mathbf{K}|\mathbf{u}(\boldsymbol{\alpha})\rangle = \|\mathbf{f}\|_2 |\mathbf{f}\rangle. \quad (13)$$

The unknowns are thus c and $\boldsymbol{\alpha}$. Once $\boldsymbol{\alpha}$ is determined, c follows. The main task is therefore to determine $\boldsymbol{\alpha}$.

2.3.2 Loss functions

To determine $\boldsymbol{\alpha}$, VQLS casts the linear-system solve as an optimization problem. A loss function measures trial-solution quality, and a classical optimizer repeatedly updates the parameters to reduce that loss. The loss functions below are evaluated at the ansatz state $|\mathbf{u}(\boldsymbol{\alpha})\rangle$. We omit its argument $\boldsymbol{\alpha}$ in the following formulas for brevity. Common choices are:

- (1) The global loss [32] measures how far $\mathbf{K}|\mathbf{u}\rangle$ departs from the direction of $|\mathbf{f}\rangle$:

$$\hat{L}_G = \langle \mathbf{u} | \mathbf{H}_G | \mathbf{u} \rangle, \quad \mathbf{H}_G = \mathbf{K}^T (\mathbf{I} - |\mathbf{f}\rangle \langle \mathbf{f}|) \mathbf{K}. \quad (14)$$

Its normalized form is

$$L_G = \frac{\hat{L}_G}{\langle \mathbf{u} | \mathbf{K}^T \mathbf{K} | \mathbf{u} \rangle} = 1 - \frac{\langle \mathbf{u} | \mathbf{K}^T |\mathbf{f}\rangle \langle \mathbf{f}| \mathbf{K} | \mathbf{u} \rangle}{\langle \mathbf{u} | \mathbf{K}^T \mathbf{K} | \mathbf{u} \rangle}. \quad (15)$$

- (2) The local loss [32] measures this departure separately on each qubit:

$$\begin{aligned} \hat{L}_L &= \langle \mathbf{u} | \mathbf{H}_L | \mathbf{u} \rangle, \quad \mathbf{H}_L = \mathbf{K}^T (\mathbf{I} - \mathbf{F} \mathbf{P} \mathbf{F}^H) \mathbf{K} \\ \mathbf{P} &= \frac{1}{n} \sum_{j=1}^n \mathbf{I}_2^{\otimes(j-1)} \otimes \boldsymbol{\tau} \otimes \mathbf{I}_2^{\otimes(n-j)}, \quad \boldsymbol{\tau} = |0\rangle \langle 0| = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix}, \end{aligned} \quad (16)$$

where $\mathbf{F}$ is the unitary defined in Eq. (4), with $\mathbf{F}|0\rangle^{\otimes n} = |\mathbf{f}\rangle$, and $\mathbf{F}^H$ is its conjugate transpose. The operator $\mathbf{P}$ is the weighted average of n single-qubit projectors $\boldsymbol{\tau} = |0\rangle \langle 0|$, which measures how far $\mathbf{F}^H \mathbf{K} |\mathbf{u}\rangle$ departs from $|0\rangle^{\otimes n}$. Its normalized form is

$$L_L = \frac{\hat{L}_L}{\langle \mathbf{u} | \mathbf{K}^T \mathbf{K} | \mathbf{u} \rangle}. \quad (17)$$

- (3) The residual loss [48] directly measures the residual of Eq. (1):

$$L_R = \|c\mathbf{K}|\mathbf{u}\rangle - \|\mathbf{f}\|_2 |\mathbf{f}\rangle\|_2^2, \quad (18)$$

where c is the scaling factor defined in Eq. (11). The normalized global and local losses both lie in $[0, 1]$, and the residual loss is the squared L_2 norm of the residual of Eq. (1).

If $\mathbf{K}$ is invertible and the ansatz can represent the normalized exact solution, the global minimum of each loss is zero. In principle, optimizing $\boldsymbol{\alpha}$ to reach that minimum yields the solution. Estimating the loss and its gradient is the main computational expense during optimization. Figure 2 shows the VQLS workflow: the quantum processor estimates the objective, and the classical optimizer updates the parameters.

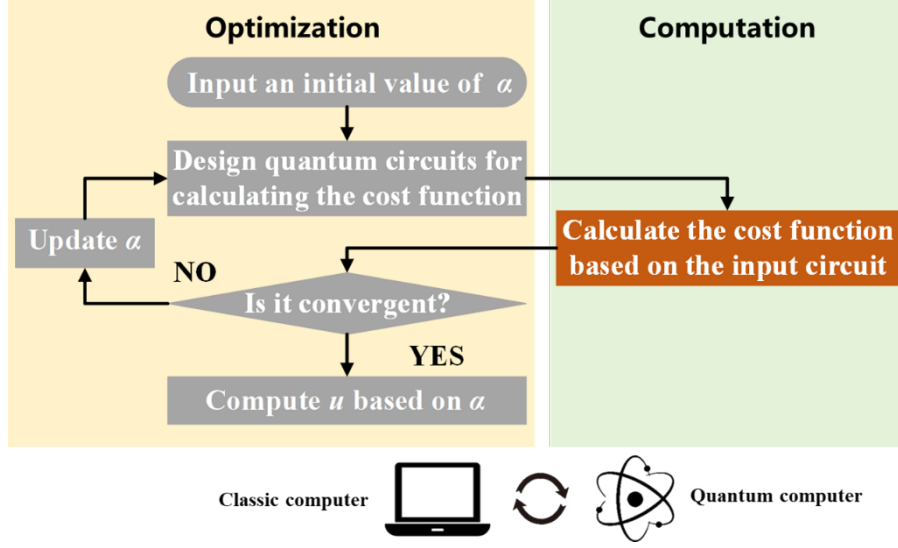


Figure 2 Hybrid quantum–classical optimization loop of VQLS

2.3.3 Hadamard test

The loss functions contain vector inner products and operator expectation values. A Hadamard test maps the real part of a unitary expectation value or of an overlap between two quantum states to the measurement probability of an ancilla qubit. Equations (14)–(18) involve $\langle \mathbf{u} | \mathbf{K}^T \mathbf{K} | \mathbf{u} \rangle$, $\langle \mathbf{f} | \mathbf{K} | \mathbf{u} \rangle$, and $\langle \mathbf{u} | \mathbf{K}^T \mathbf{F} \mathbf{P} \mathbf{F}^H \mathbf{K} | \mathbf{u} \rangle$. Explicitly forming $\mathbf{K}^T \mathbf{K}$ on a classical computer requires $O(N^2)$ storage and arithmetic in the stated setting, whereas the corresponding scalars for sparse $\mathbf{K}$ can be evaluated at a cost proportional to the number of nonzeros. In the quantum formulation, substituting Eq. (5) into the first term gives

$$\langle \mathbf{u} | \mathbf{K}^T \mathbf{K} | \mathbf{u} \rangle = \sum_{i=0}^{M-1} \sum_{j=0}^{M-1} a_i^* a_j \langle \mathbf{u} | \mathbf{U}_{i,j} | \mathbf{u} \rangle, \quad \mathbf{U}_{i,j} = \mathbf{K}_i^H \mathbf{K}_j. \quad (19)$$

Because the product $\mathbf{U}_{i,j}$ of two unitary matrices is unitary, evaluating the loss reduces to estimating its expectation value in $|\mathbf{u}\rangle$. Figure 3 shows the corresponding Hadamard-test circuit.

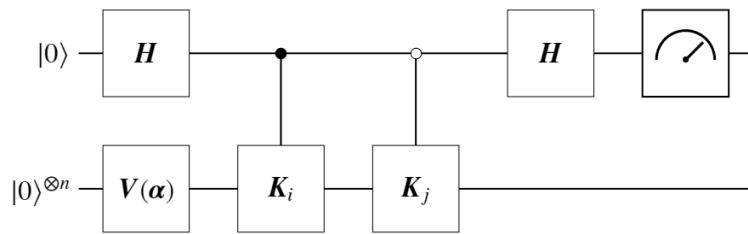


Figure 3 Hadamard-test circuit for estimating $\text{Re} \langle \mathbf{u} | \mathbf{K}_i^H \mathbf{K}_j | \mathbf{u} \rangle$

Run the circuit in Fig. 3 and denote the probability of measuring the ancilla in $|0\rangle$ by $p_{i,j}$. Then

$$\text{Re} (\langle \mathbf{u} | \mathbf{K}_i^H \mathbf{K}_j | \mathbf{u} \rangle) = 2p_{i,j} - 1. \quad (20)$$

Likewise, Eq. (5) gives

$$\langle \mathbf{f} | \mathbf{K} | \mathbf{u} \rangle = \sum_{i=0}^{M-1} a_i \langle \mathbf{f} | \mathbf{K}_i | \mathbf{u} \rangle. \quad (21)$$

If the probability of measuring the ancilla in $|0\rangle$ is β_i , then

$$\text{Re} (\langle \mathbf{f} | \mathbf{K}_i | \mathbf{u} \rangle) = 2\beta_i - 1. \quad (22)$$

These probabilities are estimated from finitely many repeated measurements. For independent Bernoulli samples, the standard error is $\sqrt{p(1-p)/S} = O(S^{-1/2})$, where p is the true probability of the outcome and S is the number of shots.

The matrix $\mathbf{K}$, vector $\mathbf{f}$, and ansatz amplitudes are real in this work, so the final objectives are real. Individual unitary terms may still have complex expectation values, but their imaginary parts cancel in the weighted sum. General complex-valued problems would require a phase gate on the ancilla to estimate the imaginary part. Note also that $\mathbf{K}|\mathbf{u}\rangle$ is generally unnormalized and cannot be prepared directly as the output of an ordinary unitary gate.

2.4 Three challenges for VQLS

When combining classical finite element discretization with VQLS, comparing only linear-system dimensions is insufficient: one must also account for data encoding, matrix representation, quantum measurement, and classical optimization. Based on Eqs. (5)–(22), we focus on three challenges.

(1) Term-by-term objective evaluation is costly. Global, local, and residual losses commonly contain $\mathbf{K}^T \mathbf{K}$. If $\mathbf{K}$ has M unitary terms, the direct double expansion in Eq. (19) contains M^2 ordered inner products. Conjugate symmetry reduces the prefactor but not the $O(M^2)$ scaling.

(2) Generic matrix representations do not exploit the structure of finite element stiffness matrices. Pauli decomposition applies to any matrix, but an n -qubit system has $4^n = N^2$ candidate Pauli strings. The number of nonzero terms depends on stiffness-matrix sparsity, degree-of-freedom ordering, and coefficient structure. Using mechanical structure may reduce the number of unitary terms in the LCU decomposition.

(3) VQLS convergence remains a challenge. The losses in Eqs. (14)–(18) are constructed from different error measures, and with many ansatz parameters their optimization can stall or fail to find a solution. In particular, the loss may become very small while the displacement error remains large.

To address these challenges, Section 3.1 constructs an LCU decomposition of the stiffness matrix using regular voxel-grid adjacency, Section 3.2 develops a variational objective with one action of $\mathbf{K}$ from the principle of minimum potential energy and relates its gradient to displacement error, and Section 4 encodes multiple unitary terms and parameter-shifted states in index registers through a block-Hadamard test, reducing the number of circuit configurations. Together, these steps define a voxel-based quantum finite element framework intended to lower quantum computational cost and improve convergence.

3. Quantum finite element method on voxel grids

Section 2 presented the general VQLS procedure. Applying VQLS to finite elements requires two further steps specific to mechanics. First, we exploit stiffness-matrix structure to reduce the number of unitary terms in the LCU decomposition. Second, we choose a variational objective consistent with the equilibrium equations. The Pauli decomposition in Section 2.2 applies to any $N \times N$ matrix but has up to $4^n = N^2$ candidate Pauli strings. Finite element stiffness matrices are assembled from local element matrices according to nodal connectivity. Their nonzero patterns therefore reflect mesh adjacency. We construct the quantum representation from the mesh and degree-of-freedom ordering rather than treating the stiffness matrix as an arbitrary dense matrix.

Voxel grids discretize a geometric domain with regularly arranged elements of identical topology. They provide uniform element connectivity, regular degree-of-freedom ordering, and convenient parallel assembly [22, 49]. For finite elements with local support, lexicographic numbering along the coordinate directions yields a banded stiffness matrix in one dimension. In two and three dimensions, the matrices admit recursive block-banded forms. This structure directly links geometric discretization to quantum operator decomposition. Local coupling between adjacent nodes produces a finite number of block-diagonal offsets. Cyclic permutation matrices handle these offsets, and the remaining blocks are decomposed into unitaries.

We use this structure to construct an LCU decomposition. The upper bound on the number of unitary terms depends on the spatial dimension and the degrees of freedom per node, but not on the node count along each coordinate direction. Section 3.1 derives the decomposition for voxel finite elements. Section 3.2 develops a variational objective from the principle of minimum potential energy in solid mechanics.

3.1 LCU decomposition of stiffness matrices on voxel grids

This section derives LCU decompositions for real symmetric stiffness matrices on one-, two-, and three-dimensional voxel grids. We begin in one dimension, where local coupling directly gives a banded matrix. The recursive block structure then extends the result to two and three dimensions. Each decomposition has three steps. A cyclic permutation moves off-diagonal blocks onto the main diagonal. Singular value decomposition is applied to the 2×2 real blocks. Euler's formula then expresses each nonnegative diagonal matrix as a sum of two diagonal unitaries. The resulting unitary terms contain cyclic permutations, 2×2 orthogonal blocks, and diagonal phases. Remark 2 and Appendices 1 and 2 describe their quantum circuits and resource assumptions.

3.1.1 LCU decomposition for a one-dimensional voxel grid

(a) One degree of freedom per node Consider a one-dimensional finite element problem with $N_x = 2^{n_x}$ nodes along the x direction, where n_x is the number of qubits needed for the node index. We first consider one degree of freedom per node, as in elementary heat-conduction and one-dimensional string-vibration problems. The finite element stiffness matrix $\mathbf{K}$ is

$$\mathbf{K} = \begin{bmatrix} k_{0,0} & k_{0,1} & & & \\ k_{1,0} & k_{1,1} & k_{1,2} & & \\ & k_{2,1} & k_{2,2} & \ddots & \\ & & \ddots & \ddots & k_{N_x-2,N_x-1} \\ & & & k_{N_x-1,N_x-2} & k_{N_x-1,N_x-1} \end{bmatrix}. \quad (23)$$

To obtain an LCU decomposition, split this stiffness matrix into three matrices:

$$\begin{aligned} \mathbf{K}_1 &= \begin{bmatrix} k_{0,0} & & & & \\ & k_{1,1} & & & \\ & & k_{2,2} & & \\ & & & \ddots & \\ & & & & k_{N_x-1,N_x-1} \end{bmatrix} = \bigoplus_{n=0}^{N_x-1} k_{n,n} \\ \mathbf{K}_2 &= \begin{bmatrix} k_{0,1} & & & & \\ & k_{1,2} & & & \\ & & \ddots & & \\ & & & \ddots & \\ & & & & k_{N_x-2,N_x-1} \end{bmatrix} = \left[\bigoplus_{n=0}^{N_x-2} k_{n,n+1} \oplus 0 \right] \mathbf{S}_{N_x} \\ \mathbf{K}_3 &= \begin{bmatrix} k_{1,0} & & & & \\ & k_{2,1} & & & \\ & & \ddots & & \\ & & & \ddots & \\ & & & & k_{N_x-1,N_x-2} \end{bmatrix} = \mathbf{S}_{N_x}^T \left[\bigoplus_{n=0}^{N_x-2} k_{n+1,n} \oplus 0 \right], \end{aligned} \quad (24)$$

where $\mathbf{S}_{N_x} = \begin{bmatrix} \mathbf{0} & \mathbf{I}_{N_x-1} \\ 1 & \mathbf{0} \end{bmatrix}$ is a cyclic permutation matrix. It is unitary and its quantum circuit and resource assumptions are given in Remark 2.

Because a product of unitaries is unitary, only the following three diagonal matrices require further decomposition:

$$\begin{aligned} \mathbf{D}_1 &= \bigoplus_{n=0}^{N_x-1} k_{n,n}, \\ \mathbf{D}_2 &= \left(\bigoplus_{n=0}^{N_x-2} k_{n,n+1} \right) \oplus 0, \end{aligned}$$

$$\mathbf{D}_3 = \left(\bigoplus_{n=0}^{N_x-2} k_{n+1,n} \right) \oplus 0.$$

For a unified derivation, denote each matrix by $\bigoplus_{n=0}^{N_x-1} \hat{k}_n$. Divide each diagonal entry by the largest absolute entry so that it lies in $[-1, 1]$. Euler's formula then expresses it as the average of two conjugate phases. Indeed, any real a with $0 \leq |a| \leq 1$ can be written as

$$a = \frac{1}{2} e^{i\phi} + \frac{1}{2} e^{-i\phi}, \quad \phi = \arccos(a) \in [0, \pi]. \quad (25)$$

By Eq. (25),

$$\bigoplus_{n=0}^{N_x-1} \hat{k}_n = \frac{k_{\max}}{2} \bigoplus_{n=0}^{N_x-1} e^{i\phi_n} + \frac{k_{\max}}{2} \bigoplus_{n=0}^{N_x-1} e^{-i\phi_n}, \quad (26)$$

where

$$k_{\max} = \max_{0 \leq n \leq N_x-1} (|\hat{k}_n|), \quad \phi_n = \arccos \left(\frac{\hat{k}_n}{k_{\max}} \right). \quad (27)$$

If $k_{\max} = 0$, the diagonal matrix is zero and should be omitted from the LCU decomposition. Equation (27) applies only when $k_{\max} > 0$.

Applying Eq. (27) to the three diagonal matrices gives the following expression. If κ_1 or κ_2 is zero, the corresponding zero matrix is omitted from the LCU decomposition.

$$\begin{aligned} \bigoplus_{n=0}^{N_x-1} k_{n,n} &= \frac{\kappa_1}{2} \left(\bigoplus_{n=0}^{N_x-1} e^{i\phi_n} + \bigoplus_{n=0}^{N_x-1} e^{-i\phi_n} \right) \\ \bigoplus_{n=0}^{N_x-2} k_{n,n+1} \oplus 0 &= \bigoplus_{n=0}^{N_x-2} k_{n+1,n} \oplus 0 = \frac{\kappa_2}{2} \left[\left(\bigoplus_{n=0}^{N_x-2} e^{i\psi_n} \right) \oplus 1 + \left(\bigoplus_{n=0}^{N_x-2} e^{-i\psi_n} \right) \oplus (-1) \right], \end{aligned} \quad (28)$$

where

$$\begin{aligned} \kappa_1 &= \max_{0 \leq n \leq N_x-1} |k_{n,n}|, \quad \phi_n = \arccos \frac{k_{n,n}}{\kappa_1} \\ \kappa_2 &= \max_{0 \leq n \leq N_x-2} |k_{n,n+1}|, \quad \psi_n = \arccos \frac{k_{n,n+1}}{\kappa_2}. \end{aligned} \quad (29)$$

Substituting Eq. (28) into Eq. (24) gives

$$\begin{aligned} \mathbf{K} &= \frac{\kappa_1}{2} \left(\bigoplus_{n=0}^{N_x-1} e^{i\phi_n} + \bigoplus_{n=0}^{N_x-1} e^{-i\phi_n} \right) + \frac{\kappa_2}{2} \left[\left(\bigoplus_{n=0}^{N_x-2} e^{i\psi_n} \right) \oplus 1 + \left(\bigoplus_{n=0}^{N_x-2} e^{-i\psi_n} \right) \oplus (-1) \right] \mathbf{S}_{N_x} \\ &\quad + \frac{\kappa_2}{2} \mathbf{S}_{N_x}^T \left[\left(\bigoplus_{n=0}^{N_x-2} e^{i\psi_n} \right) \oplus 1 + \left(\bigoplus_{n=0}^{N_x-2} e^{-i\psi_n} \right) \oplus (-1) \right]. \end{aligned} \quad (30)$$

Thus, the one-dimensional stiffness matrix is a sum of at most six unitary terms. The stated implementation time for the cyclic permutation matrix $\mathbf{S}_{N_x}$ is $O(\text{poly}(\log N))$, and the other unitaries are diagonal and can be implemented with quantum multiplexors [52, 53], also with the stated $O(\text{poly}(\log N))$ time complexity. See Remark 2 and Appendix 2 for the associated assumptions.

(b) Two degrees of freedom per node Next, consider two degrees of freedom per node, such as the deflection and rotation at each node of an Euler–Bernoulli beam element. For $N_x = 2^{n_x}$ nodes along x and $d = 2$ degrees of freedom per node, the total number of degrees of freedom is $N = dN_x$, and $\mathbf{K}$ has the following block-tridiagonal form:

$$\mathbf{K} = \begin{bmatrix} \mathbf{k}_{0,0} & \mathbf{k}_{0,1} & & & \\ \mathbf{k}_{1,0} & \mathbf{k}_{1,1} & \mathbf{k}_{1,2} & & \\ & \mathbf{k}_{2,1} & \mathbf{k}_{2,2} & \ddots & \\ & & \ddots & \ddots & \mathbf{k}_{N_x-2,N_x-1} \\ & & & \mathbf{k}_{N_x-1,N_x-2} & \mathbf{k}_{N_x-1,N_x-1} \end{bmatrix}, \quad (31)$$

where each submatrix $\mathbf{k}_{i,j}$ is 2×2 . This block structure facilitates quantum encoding: n_x qubits encode the node index, and another $\log_2 d$ qubits encode the degrees of freedom at each node (one qubit for $d = 2$). For the LCU decomposition, write $\mathbf{K}$ as

$$\mathbf{K} = \mathbf{K}_1 + \mathbf{K}_2 + \mathbf{K}_3, \quad (32)$$

where

$$\begin{aligned} \mathbf{K}_1 &= \begin{bmatrix} \mathbf{k}_{0,0} & & & & \\ & \mathbf{k}_{1,1} & & & \\ & & \mathbf{k}_{2,2} & & \\ & & & \ddots & \\ & & & & \mathbf{k}_{N_x-1,N_x-1} \end{bmatrix} = \bigoplus_{n=0}^{N_x-1} \mathbf{k}_{n,n} \\ \mathbf{K}_2 &= \begin{bmatrix} \mathbf{k}_{0,1} & & & & \\ & \mathbf{k}_{1,2} & & & \\ & & \ddots & & \\ & & & \mathbf{k}_{N_x-2,N_x-1} & \\ & & & & \end{bmatrix} = \left[\bigoplus_{n=0}^{N_x-2} \mathbf{k}_{n,n+1} \oplus \mathbf{0}_2 \right] [\mathbf{S}_{N_x} \otimes \mathbf{I}_d] \\ \mathbf{K}_3 &= \begin{bmatrix} & \mathbf{k}_{1,0} & & & \\ & & \mathbf{k}_{2,1} & & \\ & & & \ddots & \\ & & & & \mathbf{k}_{N_x-1,N_x-2} \end{bmatrix} = [\mathbf{S}_{N_x}^T \otimes \mathbf{I}_d] \left[\bigoplus_{n=0}^{N_x-2} \mathbf{k}_{n+1,n} \oplus \mathbf{0}_2 \right], \end{aligned} \quad (33)$$

where $\mathbf{0}_2$ and $\mathbf{I}_d$ are the 2×2 zero matrix and the $d \times d$ identity matrix, respectively. Since $\mathbf{S}_{N_x}$ is unitary, only three block-diagonal matrices with 2×2 blocks need decomposition: $\bigoplus_{n=0}^{N_x-1} \mathbf{k}_{n,n}$, $(\bigoplus_{n=0}^{N_x-2} \mathbf{k}_{n,n+1}) \oplus \mathbf{0}_2$, and $(\bigoplus_{n=0}^{N_x-2} \mathbf{k}_{n+1,n}) \oplus \mathbf{0}_2$. Denote any of them by $\bigoplus_{n=0}^{N_x-1} \hat{\mathbf{k}}_n$. The singular value decomposition of each block is

$$\hat{\mathbf{k}}_n = \mathbf{U}_n \Sigma_n \mathbf{V}_n^T, \quad \Sigma_n = \begin{bmatrix} \sigma_{1,n} & \\ & \sigma_{2,n} \end{bmatrix}. \quad (34)$$

The formulas for $\mathbf{U}_n$, $\mathbf{V}_n$, and the singular values $\sigma_{1,n} \geq \sigma_{2,n} \geq 0$ are given in Lemma 1 of Appendix 1. Substituting Eq. (34) into $\bigoplus_{n=0}^{N_x-1} \hat{\mathbf{k}}_n$ gives

$$\bigoplus_{n=0}^{N_x-1} \hat{\mathbf{k}}_n = \left(\bigoplus_{n=0}^{N_x-1} \mathbf{U}_n \right) \left(\bigoplus_{n=0}^{N_x-1} \Sigma_n \right) \left(\bigoplus_{n=0}^{N_x-1} \mathbf{V}_n^T \right). \quad (35)$$

Both $\bigoplus_{n=0}^{N_x-1} \mathbf{U}_n$ and $\bigoplus_{n=0}^{N_x-1} \mathbf{V}_n^T$ are unitary, so completing the LCU decomposition of $\bigoplus_{n=0}^{N_x-1} \hat{\mathbf{k}}_n$ requires only the decomposition of $\bigoplus_{n=0}^{N_x-1} \text{diag}(\sigma_{1,n}, \sigma_{2,n})$. By Eq. (25), this matrix can be written as

$$\bigoplus_{n=0}^{N_x-1} \begin{bmatrix} \sigma_{1,n} & \\ & \sigma_{2,n} \end{bmatrix} = \frac{\sigma_{\max}}{2} \bigoplus_{n=0}^{N_x-1} \begin{bmatrix} e^{i\phi_{1,n}} & \\ & e^{i\phi_{2,n}} \end{bmatrix} + \frac{\sigma_{\max}}{2} \bigoplus_{n=0}^{N_x-1} \begin{bmatrix} e^{-i\phi_{1,n}} & \\ & e^{-i\phi_{2,n}} \end{bmatrix}, \quad (36)$$

where

$$\sigma_{\max} = \max_{0 \leq n \leq N_x-1} \{\sigma_{1,n}, \sigma_{2,n}\}, \quad \phi_{1,n} = \arccos \frac{\sigma_{1,n}}{\sigma_{\max}}, \quad \phi_{2,n} = \arccos \frac{\sigma_{2,n}}{\sigma_{\max}}. \quad (37)$$

If $\sigma_{\max} = 0$, all 2×2 blocks vanish and this component can be omitted, whereas the following expression assumes $\sigma_{\max} > 0$. Substituting Eq. (36) into Eq. (35) completes the LCU decomposition of $\bigoplus_{n=0}^{N_x-1} \hat{\mathbf{k}}_n$:

$$\begin{aligned} \bigoplus_{n=0}^{N_x-1} \hat{\mathbf{k}}_n &= \frac{\sigma_{\max}}{2} \left(\bigoplus_{n=0}^{N_x-1} \mathbf{U}_n \right) \left(\bigoplus_{n=0}^{N_x-1} \begin{bmatrix} e^{i\phi_{1,n}} & \\ & e^{i\phi_{2,n}} \end{bmatrix} \right) \left(\bigoplus_{n=0}^{N_x-1} \mathbf{V}_n^T \right) \\ &+ \frac{\sigma_{\max}}{2} \left(\bigoplus_{n=0}^{N_x-1} \mathbf{U}_n \right) \left(\bigoplus_{n=0}^{N_x-1} \begin{bmatrix} e^{-i\phi_{1,n}} & \\ & e^{-i\phi_{2,n}} \end{bmatrix} \right) \left(\bigoplus_{n=0}^{N_x-1} \mathbf{V}_n^T \right). \end{aligned} \quad (38)$$

Thus, any block-diagonal matrix $\bigoplus_{n=0}^{N_x-1} \hat{\mathbf{k}}_n$ with 2×2 blocks is a linear combination of two unitaries. Applying the same steps as in Eqs. (34)–(38) to $\mathbf{K}_1$, $\mathbf{K}_2$, and $\mathbf{K}_3$ gives two unitaries for each, so the one-dimensional voxel-grid stiffness matrix requires at most six unitary terms for $d = 2$.

The unitaries in Eq. (38) are block-diagonal matrices with 2×2 unitary blocks. In particular, $\mathbf{U}_n$ and $\mathbf{V}_n$ are 2×2 orthogonal matrices and can be represented by elementary rotation gates, as detailed in Appendix 1.

(c) Four degrees of freedom per node Finally, consider $d = 4$, for which $N = 4N_x$. A model with three degrees of freedom per node can be extended by adding one auxiliary degree of freedom with zero load and a positive diagonal entry γ . This produces a 4×4 nodal block, preserves the solution on the original three degrees of freedom, and avoids a zero eigenvalue.

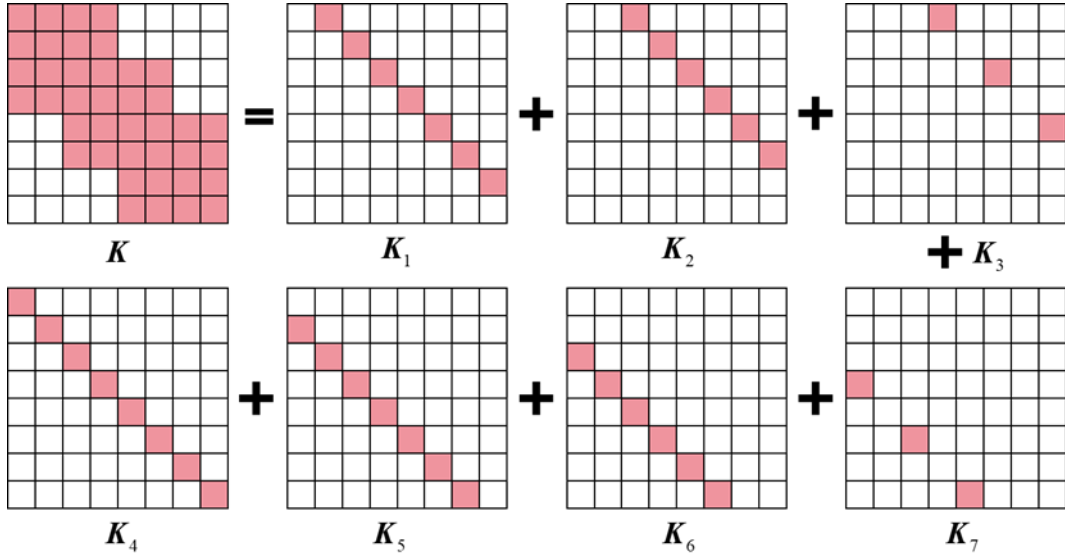


Figure 4 Rearrangement of a block-tridiagonal matrix with 4×4 blocks into a block-heptadiagonal matrix with 2×2 blocks

With four degrees of freedom per node, $\mathbf{K}$ remains the block-tridiagonal matrix in Eq. (31), but each block is 4×4 . As shown in Fig. 4, rearrange it as the sum of seven matrices with 2×2 blocks:

$$\mathbf{K} = \mathbf{K}_1 + \mathbf{K}_2 + \mathbf{K}_3 + \mathbf{K}_4 + \mathbf{K}_5 + \mathbf{K}_6 + \mathbf{K}_7, \quad (39)$$

where

$$\begin{aligned}
\mathbf{K}_1 &= \bigoplus_{n=0}^{2N_x-1} \begin{bmatrix} k_{2n,2n} & k_{2n,2n+1} \\ k_{2n+1,2n} & k_{2n+1,2n+1} \end{bmatrix}, \\
\mathbf{K}_2 &= \left[\bigoplus_{n=0}^{2N_x-2} \begin{bmatrix} k_{2n,2n+2} & k_{2n,2n+3} \\ k_{2n+1,2n+2} & k_{2n+1,2n+3} \end{bmatrix} \oplus \mathbf{0}_2 \right] [\mathbf{S}_{2N_x} \otimes \mathbf{I}_2], \\
\mathbf{K}_3 &= [\mathbf{S}_{2N_x}^T \otimes \mathbf{I}_2] \left[\bigoplus_{n=0}^{2N_x-2} \begin{bmatrix} k_{2n+2,2n} & k_{2n+2,2n+1} \\ k_{2n+3,2n} & k_{2n+3,2n+1} \end{bmatrix} \oplus \mathbf{0}_2 \right], \\
\mathbf{K}_4 &= \left[\bigoplus_{n=0}^{2N_x-3} \begin{bmatrix} k_{2n,2n+4} & k_{2n,2n+5} \\ k_{2n+1,2n+4} & k_{2n+1,2n+5} \end{bmatrix} \oplus \mathbf{0}_4 \right] [\mathbf{S}_{2N_x}^2 \otimes \mathbf{I}_2], \\
\mathbf{K}_5 &= [(\mathbf{S}_{2N_x}^T)^2 \otimes \mathbf{I}_2] \left[\bigoplus_{n=0}^{2N_x-3} \begin{bmatrix} k_{2n+4,2n} & k_{2n+4,2n+1} \\ k_{2n+5,2n} & k_{2n+5,2n+1} \end{bmatrix} \oplus \mathbf{0}_4 \right], \\
\mathbf{K}_6 &= \left[\bigoplus_{n=0}^{2N_x-4} \begin{bmatrix} k_{2n,2n+6} & k_{2n,2n+7} \\ k_{2n+1,2n+6} & k_{2n+1,2n+7} \end{bmatrix} \oplus \mathbf{0}_6 \right] [\mathbf{S}_{2N_x}^3 \otimes \mathbf{I}_2], \\
\mathbf{K}_7 &= [(\mathbf{S}_{2N_x}^T)^3 \otimes \mathbf{I}_2] \left[\bigoplus_{n=0}^{2N_x-4} \begin{bmatrix} k_{2n+6,2n} & k_{2n+6,2n+1} \\ k_{2n+7,2n} & k_{2n+7,2n+1} \end{bmatrix} \oplus \mathbf{0}_6 \right], \tag{40}
\end{aligned}$$

where $\mathbf{S}_{2N_x} = \begin{bmatrix} \mathbf{I}_{2N_x-1} \\ 1 \end{bmatrix}$ is a cyclic permutation matrix. The matrices $\mathbf{0}_2$, $\mathbf{0}_4$, and $\mathbf{0}_6$ are zero matrices of sizes 2×2 , 4×4 , and 6×6 , respectively. As in the $d = 2$ case, each $\mathbf{K}_i$ is a linear combination of two unitaries. Thus, for $d = 4$, the one-dimensional voxel-grid stiffness matrix has an LCU decomposition with 14 unitary terms.

3.1.2 LCU decomposition for two- and three-dimensional voxel grids

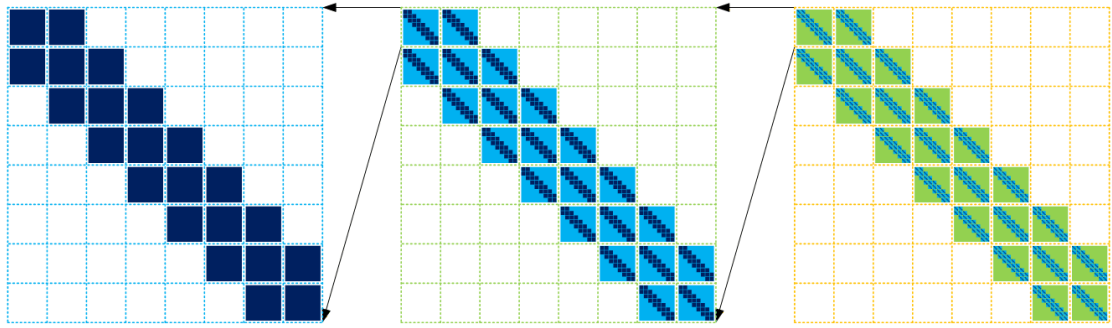


Figure 5 Recursive block-banded structure of voxel-grid stiffness matrices

A stiffness matrix assembled on a voxel grid has a self-similar block structure, as shown in Fig. 5: the three-dimensional matrix can be viewed as a block-tridiagonal matrix of two-dimensional voxel-grid stiffness matrices, and each two-dimensional matrix is likewise block tridiagonal with one-dimensional voxel-grid stiffness matrices as blocks. This property extends the decomposition in Section 3.1.1 to two and three dimensions.

(a) Two-dimensional case For a two-dimensional voxel-grid stiffness matrix $\mathbf{K}$, the block-tridiagonal structure gives

$$\mathbf{K} = \mathbf{K}_1^{(1)} + \mathbf{K}_2^{(1)} + \mathbf{K}_3^{(1)}, \tag{41}$$

where $\mathbf{K}_1^{(1)}$ contains the main-diagonal blocks of $\mathbf{K}$, whereas $\mathbf{K}_2^{(1)}$ and $\mathbf{K}_3^{(1)}$ contain the blocks immediately above and below the main diagonal, respectively. As in Eqs. (24) and (33), the latter two matrices are products of block-diagonal matrices and cyclic permutation matrices:

$$\mathbf{K}_2^{(1)} = \left[\bigoplus_{n=0}^{N_y-2} \mathbf{B}_{n,n+1} \oplus \mathbf{0}_{dN_x} \right] (\mathbf{S}_{N_y} \otimes \mathbf{I}_{dN_x}), \quad \mathbf{K}_3^{(1)} = (\mathbf{S}_{N_y} \otimes \mathbf{I}_{dN_x})^T \left[\bigoplus_{n=0}^{N_y-2} \mathbf{B}_{n+1,n} \oplus \mathbf{0}_{dN_x} \right], \quad (42)$$

where $N_y = 2^{n_y}$ is the number of nodes along y , and $\mathbf{B}_{n,n+1}$ and $\mathbf{B}_{n+1,n}$ are $dN_x \times dN_x$ coupling blocks between adjacent nodal rows. The main-diagonal and coupling blocks remain block banded along x , so the decomposition in Section 3.1.1 applies. Figure 6 illustrates the two-dimensional splitting.

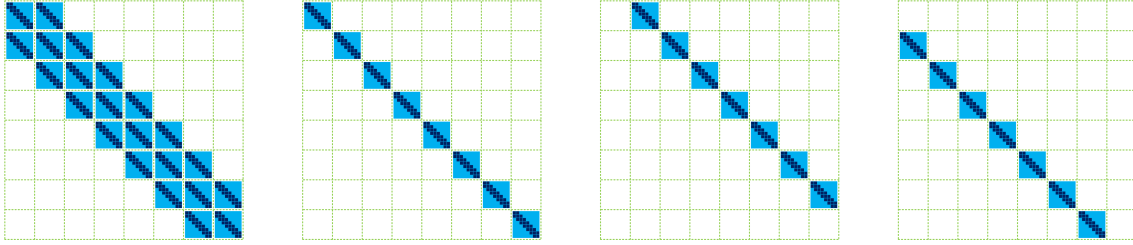


Figure 6 Three-part decomposition of a two-dimensional voxel-grid stiffness matrix

First, split the block-tridiagonal matrix into three parts along y . Then apply the one-dimensional decomposition along x to each part. The y direction produces three classes of block-diagonal offsets. Each class yields at most six unitary terms for $d = 1$ or 2, or 14 terms for $d = 4$. Thus, the two-dimensional stiffness matrix requires at most $3 \times 6 = 18$ terms for $d = 1$ or 2, and $3 \times 14 = 42$ terms for $d = 4$.

(b) Three-dimensional case The three-dimensional decomposition is analogous. First, split the voxel-grid stiffness matrix as

$$\mathbf{K} = \mathbf{K}_1^{(2)} + \mathbf{K}_2^{(2)} + \mathbf{K}_3^{(2)}, \quad (43)$$

where $\mathbf{K}_1^{(2)}$, $\mathbf{K}_2^{(2)}$, and $\mathbf{K}_3^{(2)}$ contain the main-, upper-, and lower-diagonal blocks, respectively. Each top-level block has the structure of a two-dimensional voxel-grid stiffness matrix in the x - y plane. It can therefore be split into three components as in Eq. (41). Each two-dimensional component is finally reduced to a one-dimensional decomposition from Section 3.1.1. Equivalently, the three-dimensional decomposition introduces a three-part split in each of the z and y directions. Its upper bound on unitary terms is 3^2 times the one-dimensional bound: $9 \times 6 = 54$ for $d = 1$ or 2, and $9 \times 14 = 126$ for $d = 4$.

Table 2 lists the upper bounds on the number of unitary terms in the LCU decomposition of $\mathbf{K}$ for different spatial dimensions and nodal degrees of freedom. Unlike a generic Pauli decomposition, the bound for the proposed voxel-grid decomposition does not increase with the matrix dimension when the spatial dimension and d are fixed. It depends on the spatial dimension and d . Consequently, the potential reduction relative to a Pauli decomposition can grow with problem size.

Table 2 Upper bounds on the number of unitary terms for voxel-grid stiffness matrices

	One-dimensional	Two-dimensional	Three-dimensional
$d = 1$ or 2	6	18	54
$d = 4$	14	42	126

Remark 1 Section 3.1 used three techniques: cyclic permutations to handle block-diagonal offsets, singular value decomposition of real 2×2 blocks, and Euler’s formula to express a nonnegative diagonal matrix as the sum of two diagonal unitaries. The construction can extend to other matrices with known finite block bandwidth, but the term count then depends on the block bandwidth and block size and does not imply a constant term count for arbitrary unstructured meshes.

Remark 2 The unitary terms obtained here mainly comprise cyclic permutations, 2×2 orthogonal blocks, and diagonal unitaries. A cyclic permutation can be implemented with a quantum Fourier transform or a reversible addition circuit. The gate count and depth depend on the gate set and whether an approximate QFT is used. A block-diagonal R_y operator, $\bigoplus_{i=0}^{N-1} \mathbf{R}_y(\theta_i)$,

can be expressed by a change of basis as $(\mathbf{I}_N \otimes \xi) \bigoplus_{i=0}^{N-1} \mathbf{R}_z(\theta_i) (\mathbf{I}_N \otimes \xi)^H$, where $\xi = \frac{1}{\sqrt{2}} \begin{bmatrix} -i & i \\ 1 & 1 \end{bmatrix}$ is a single-qubit unitary. Thus,

implementing the unitaries in this decomposition ultimately depends on efficient implementations of diagonal unitaries. In general, such a unitary has the form $\bigoplus_{i=0}^{2^n-1} e^{i\theta_i}$. A standard approach uses quantum multiplexors [52, 53]. Appendix 2 states that even in the worst case, when all θ_i differ, this operator can be implemented with $O(N)$ qubits and an $O(\log N)$ -time quantum circuit under the data-access assumptions stated in Appendix 2.

3.2 Solving finite element stiffness equations by minimum potential energy

Section 2 described variational quantum optimization and its convergence challenges. The loss functions in Section 2.3.2 are based on different error measures. With many ansatz parameters, optimization with these loss functions may converge slowly or fail to find a solution. In particular, the loss can become small while the displacement error remains large. Steepest descent and conjugate gradient are common classical finite element solvers. Both can be viewed as optimization methods. For a symmetric positive-definite stiffness matrix, the principle of minimum potential energy recasts the stiffness equation (1) as

$$\Pi(\mathbf{u}) = \frac{1}{2} \mathbf{u}^T \mathbf{K} \mathbf{u} - \mathbf{f}^T \mathbf{u}. \quad (44)$$

Solvers based on minimum potential energy, particularly conjugate gradient, have been highly successful in classical finite element analysis. This motivates using discrete total potential energy as a variational objective in place of a residual-based VQLS loss. The ansatz parameters are then optimized by minimizing that energy. Previous studies [24, 34] showed that, for a given relative displacement-error target, estimating the normalized energy difference can require less precision than estimating a VQLS loss. We therefore adopt a minimum-potential-energy variational algorithm and briefly describe its implementation.

Substituting Eq. (11) into Eq. (44) gives

$$\Pi(c, \boldsymbol{\alpha}) = \frac{c^2}{2} \langle \mathbf{u}(\boldsymbol{\alpha}) | \mathbf{K} | \mathbf{u}(\boldsymbol{\alpha}) \rangle - c \|\mathbf{f}\|_2 \langle \mathbf{f} | \mathbf{u}(\boldsymbol{\alpha}) \rangle. \quad (45)$$

The problem becomes $\min_{c \geq 0, \boldsymbol{\alpha} \in \Omega_{\boldsymbol{\alpha}}} \Pi(c, \boldsymbol{\alpha})$, where $\Omega_{\boldsymbol{\alpha}}$ is the ansatz-parameter domain.

Several classical optimizers can solve this problem. Gradient-based methods, including steepest descent and sequential quadratic programming (SQP), require the gradient $(\partial \Pi / \partial c, \partial \Pi / \partial \alpha_1, \dots, \partial \Pi / \partial \alpha_{N_{\boldsymbol{\alpha}}})^T$. It can be calculated analytically as follows:

$$\begin{cases} \frac{\partial \Pi}{\partial c} = c \langle \mathbf{u}(\boldsymbol{\alpha}) | \mathbf{K} | \mathbf{u}(\boldsymbol{\alpha}) \rangle - \|\mathbf{f}\|_2 \langle \mathbf{f} | \mathbf{u}(\boldsymbol{\alpha}) \rangle \\ \frac{\partial \Pi}{\partial \alpha_j} = c^2 \langle \mathbf{u}(\boldsymbol{\alpha}) | \mathbf{K} | \mathbf{u}_{\alpha_j}(\boldsymbol{\alpha}) \rangle - c \|\mathbf{f}\|_2 \langle \mathbf{f} | \mathbf{u}_{\alpha_j}(\boldsymbol{\alpha}) \rangle, \quad 1 \leq j \leq N_{\boldsymbol{\alpha}} \end{cases}, \quad (46)$$

where

$$|\mathbf{u}_{\alpha_j}(\boldsymbol{\alpha})\rangle = \frac{d|\mathbf{u}(\boldsymbol{\alpha})\rangle}{d\alpha_j} = \frac{d\mathbf{V}(\boldsymbol{\alpha})}{d\alpha_j} |0\rangle^{\otimes n} = \mathbf{V}_{\alpha_j} |0\rangle^{\otimes n}. \quad (47)$$

If the ansatz in Fig. 1 is used, each parameter occurs in exactly one R_y gate. Since $d\mathbf{R}_y(\alpha)/d\alpha = \mathbf{R}_y(\alpha + \pi)/2$, we obtain

$$\mathbf{V}_{\alpha_j}(\boldsymbol{\alpha}) = \frac{1}{2} \mathbf{V}(\boldsymbol{\alpha}_{j,\pi}), \quad \boldsymbol{\alpha}_{j,\pi} = (\alpha_1, \alpha_2, \dots, \alpha_j + \pi, \dots, \alpha_{N_{\boldsymbol{\alpha}}})^T. \quad (48)$$

Substituting Eq. (48) into the derivative of the total potential energy gives

$$\begin{cases} \frac{\partial \Pi}{\partial c} = c \langle \mathbf{u}(\boldsymbol{\alpha}) | \mathbf{K} | \mathbf{u}(\boldsymbol{\alpha}) \rangle - \|\mathbf{f}\|_2 \langle \mathbf{f} | \mathbf{u}(\boldsymbol{\alpha}) \rangle \\ \frac{\partial \Pi}{\partial \alpha_j} = \frac{1}{2} [c^2 \langle \mathbf{u}(\boldsymbol{\alpha}) | \mathbf{K} | \mathbf{u}(\boldsymbol{\alpha}_{j,\pi}) \rangle - c \|\mathbf{f}\|_2 \langle \mathbf{f} | \mathbf{u}(\boldsymbol{\alpha}_{j,\pi}) \rangle], \quad 1 \leq j \leq N_{\boldsymbol{\alpha}} \end{cases}. \quad (49)$$

Alternatively, a forward finite difference approximates the gradient:

$$\begin{cases} \frac{\partial \Pi}{\partial c} = c \langle \mathbf{u}(\boldsymbol{\alpha}) | \mathbf{K} | \mathbf{u}(\boldsymbol{\alpha}) \rangle - \|\mathbf{f}\|_2 \langle \mathbf{f} | \mathbf{u}(\boldsymbol{\alpha}) \rangle \\ \frac{\partial \Pi}{\partial \alpha_j} \approx \frac{\Pi(c, \boldsymbol{\alpha}_{j,\Delta\alpha}) - \Pi(c, \boldsymbol{\alpha})}{\Delta\alpha} \end{cases}, \quad (50)$$

where $\boldsymbol{\alpha}_{j,\Delta\alpha} = (\alpha_1, \dots, \alpha_j + \Delta\alpha, \dots, \alpha_{N_\alpha})^T$.

Treat $\mathbf{s} = (c, \boldsymbol{\alpha}^T)^T$ as the unknown parameters, and write the displacement vector as $\mathbf{u}(\mathbf{s})$. Define its Jacobian as

$$\mathbf{J}(\mathbf{s}) = \frac{\partial \mathbf{u}(\mathbf{s})}{\partial \mathbf{s}^T} \in \mathbb{R}^{N \times (N_\alpha + 1)}.$$

If $\mathbf{J}(\mathbf{s})$ has full row rank, so that its smallest singular value $\sigma_{\min}(\mathbf{J}(\mathbf{s})) > 0$, the following error bounds hold:

$$\frac{\|\nabla_s \Pi(\mathbf{s})\|_2}{\sigma_{\max}(\mathbf{J}(\mathbf{s})) \lambda_{\max}(\mathbf{K})} \leq \|\mathbf{u}(\mathbf{s}) - \mathbf{K}^{-1} \mathbf{f}\|_2 \leq \frac{\|\nabla_s \Pi(\mathbf{s})\|_2}{\sigma_{\min}(\mathbf{J}(\mathbf{s})) \lambda_{\min}(\mathbf{K})}, \quad (51)$$

where $\lambda_{\min}$ and $\lambda_{\max}$ are the smallest and largest eigenvalues of $\mathbf{K}$, while $\sigma_{\min}$ and $\sigma_{\max}$ are the smallest and largest singular values of $\mathbf{J}$. The bounds show how the gradient and ansatz Jacobian jointly affect displacement error. When the Jacobian has full row rank, the gradient norm can indicate the solution error. Moreover, $\sigma_{\min}(\mathbf{J})$ reflects, to some extent, the sensitivity of the ansatz to parameter changes. A smaller value indicates weaker local sensitivity and can make small gradients and slow convergence more likely. A larger value generally improves local identifiability of the ansatz parameters and can assist optimization.

Minimizing the total potential energy over $\mathbf{s} = (c, \boldsymbol{\alpha}^T)^T$ mainly requires $\langle \mathbf{u}(\boldsymbol{\alpha}) | \mathbf{K} | \mathbf{u}(\boldsymbol{\alpha}) \rangle$, $\langle \mathbf{f} | \mathbf{u}(\boldsymbol{\alpha}) \rangle$, and $\langle \mathbf{u}(\boldsymbol{\alpha}) | \mathbf{K} | \mathbf{u}(\boldsymbol{\alpha}_{j,\pi}) \rangle$. The load-state overlap $\langle \mathbf{f} | \mathbf{u}(\boldsymbol{\alpha}) \rangle$ can be estimated directly with a Hadamard test. Applying the LCU decomposition in Eq. (5) to $\mathbf{K}$ gives

$$\begin{aligned} \langle \mathbf{u}(\boldsymbol{\alpha}) | \mathbf{K} | \mathbf{u}(\boldsymbol{\alpha}) \rangle &= \sum_{i=0}^{M-1} a_i \langle \mathbf{u}(\boldsymbol{\alpha}) | \mathbf{K}_i | \mathbf{u}(\boldsymbol{\alpha}) \rangle, \\ \langle \mathbf{u}(\boldsymbol{\alpha}) | \mathbf{K} | \mathbf{u}(\boldsymbol{\alpha}_{j,\pi}) \rangle &= \sum_{i=0}^{M-1} a_i \langle \mathbf{u}(\boldsymbol{\alpha}) | \mathbf{K}_i | \mathbf{u}(\boldsymbol{\alpha}_{j,\pi}) \rangle. \end{aligned} \quad (52)$$

These terms are estimated by the block-Hadamard circuits in Section 4.

4. Block-Hadamard quantum test

Estimating Eq. (52) with the Hadamard test from Section 2 requires a separate test for each $\mathbf{K}_i$. Each potential-energy or derivative term therefore needs $O(M)$ controlled-unitary circuit configurations. Evaluating all variational parameters increases this number with N_α . This section proposes a block-Hadamard quantum test. A quantum multiplexor coherently selects several $\mathbf{K}_i$ within one circuit. The ancilla measurement probability then contains a weighted sum of overlaps. This enables batched estimation of $\langle \mathbf{u} | \mathbf{K}_i | \mathbf{u} \rangle$ and $\langle \mathbf{u} | \mathbf{K}_i | \mathbf{u}_{\alpha_j} \rangle$.

4.1 Block-Hadamard tests for potential energy and its derivatives

Let $\mathbf{V}_0 = \mathbf{V}(\boldsymbol{\alpha})$ and $|\mathbf{u}_0\rangle = |\mathbf{u}(\boldsymbol{\alpha})\rangle$. For $j = 1, \dots, N_\alpha$, define the normalized parameter-shifted state

$$\mathbf{V}_j = \mathbf{V}(\boldsymbol{\alpha}_{j,\pi}), \quad |\mathbf{u}_j\rangle = \mathbf{V}_j |0\rangle^{\otimes n}.$$

By Eq. (48), the derivative state is $|\mathbf{u}_{\alpha_j}\rangle = |\mathbf{u}_j\rangle/2$. The quantum circuit implements only the unitary $\mathbf{V}_j$. The factor 1/2 is retained in the classical postprocessing of Eq. (49). The loss functions in Section 2.3.2 typically require overlaps of the form $\langle \mathbf{u} | \mathbf{K}_i^H \mathbf{K}_j | \mathbf{u} \rangle$, involving $O(M^2)$ circuit calls when evaluated term by term. By contrast, the total-potential-energy objective from Section 3.2 requires only $\langle \mathbf{u} | \mathbf{K}_i | \mathbf{u} \rangle$ and $\langle \mathbf{u} | \mathbf{K}_i | \mathbf{u}_{\alpha_j} \rangle$. Their term-by-term evaluation uses $O(M)$ calls. Measurement estimates also have sampling error of order $O(S^{-1/2})$, where S is the number of shots. More overlaps therefore increase the total number of shots needed for a given precision. We now derive block-Hadamard tests for the two required overlap types. First, consider $\langle \mathbf{u} | \mathbf{K} | \mathbf{u}_j \rangle$:

$$\langle \mathbf{u} | \mathbf{K} | \mathbf{u}_j \rangle = \sum_{i=0}^{M-1} a_i \langle \mathbf{u} | \mathbf{K}_i | \mathbf{u}_j \rangle. \quad (53)$$

A term-by-term Hadamard test for the M overlaps on the right-hand side of Eq. (53) requires M circuit configurations, each sampled repeatedly. Instead, group the terms into blocks:

$$\langle \mathbf{u} | \mathbf{K} | \mathbf{u}_j \rangle = \sum_{i=0}^{B-1} a_i \langle \mathbf{u} | \mathbf{K}_i | \mathbf{u}_j \rangle + \sum_{i=B}^{2B-1} a_i \langle \mathbf{u} | \mathbf{K}_i | \mathbf{u}_j \rangle + \cdots + \sum_{i=M-B}^{M-1} a_i \langle \mathbf{u} | \mathbf{K}_i | \mathbf{u}_j \rangle, \quad (54)$$

where $M = 2^m$ and $B = 2^b$.

We now design a circuit that evaluates the first block in Eq. (54), $\sum_{i=0}^{B-1} a_i \langle \mathbf{u} | \mathbf{K}_i | \mathbf{u}_j \rangle$, in one batch. We consider (a) nonnegative real coefficients a_i and (b) general complex coefficients.

(a) a_i are nonnegative real numbers First, suppose all a_i are nonnegative real numbers. Amplitude-encode the coefficients $(\sqrt{a_0}, \dots, \sqrt{a_{B-1}})$ of the first block:

$$|\hat{\mathbf{a}}_1\rangle = \frac{1}{\sqrt{\sum_{i=0}^{B-1} a_i}} \sum_{i=0}^{B-1} \sqrt{a_i} |i\rangle = \hat{\mathbf{A}}_1 |0\rangle^{\otimes b}. \quad (55)$$

Figure 7 shows the circuit for $B = 4$. Here, $\mathbf{V}_j$ is the state-preparation unitary for the normalized parameter-shifted state, not the nonunitary derivative $\partial \mathbf{V} / \partial \alpha_j$.

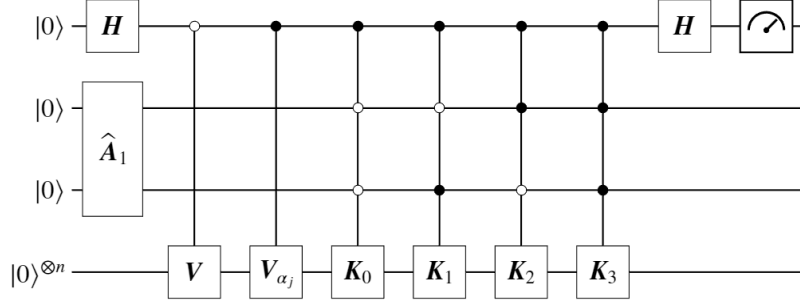


Figure 7 Block-Hadamard circuit for the weighted sum in the first block with nonnegative real coefficients ($B = 4$)

The input state is $|\psi\rangle = |0\rangle|0\rangle^{\otimes b}|0\rangle^{\otimes n}$. After the first Hadamard gate, it becomes $|\psi\rangle = (|0\rangle + |1\rangle)|\hat{\mathbf{a}}_1\rangle|0\rangle^{\otimes n}/\sqrt{2}$. The next two layers are controlled. When the first qubit is $|0\rangle$, $\mathbf{V}(\boldsymbol{\alpha})$ acts on the n -qubit register. When it is $|1\rangle$, $\mathbf{V}_j$ acts on that register. The state is then

$$|\psi\rangle = \frac{1}{\sqrt{2}} |0\rangle |\hat{\mathbf{a}}_1\rangle |\mathbf{u}\rangle + \frac{1}{\sqrt{2}} |1\rangle |\hat{\mathbf{a}}_1\rangle |\mathbf{u}_j\rangle. \quad (56)$$

After the quantum multiplexors, the state becomes

$$|\psi\rangle = \frac{1}{\sqrt{2}} \frac{1}{\sqrt{\sum_{i=0}^{B-1} a_i}} \left[|0\rangle \sum_{i=0}^{B-1} \sqrt{a_i} |i\rangle |\mathbf{u}\rangle + |1\rangle \sum_{i=0}^{B-1} \sqrt{a_i} |i\rangle \mathbf{K}_i |\mathbf{u}_j\rangle \right]. \quad (57)$$

Finally, the Hadamard gate produces

$$|\psi\rangle = \frac{1}{2} \frac{1}{\sqrt{\sum_{i=0}^{B-1} a_i}} \left[|0\rangle \left(\sum_{i=0}^{B-1} \sqrt{a_i} |i\rangle (|\mathbf{u}\rangle + \mathbf{K}_i |\mathbf{u}_j\rangle) \right) + |1\rangle \left(\sum_{i=0}^{B-1} \sqrt{a_i} |i\rangle (|\mathbf{u}\rangle - \mathbf{K}_i |\mathbf{u}_j\rangle) \right) \right]. \quad (58)$$

Measure the first qubit and denote the probabilities of $|0\rangle$ and $|1\rangle$ by ζ_0 and ζ_1 , respectively. Then

$$\zeta_0 = \frac{1}{2} \left[1 + \frac{1}{\sum_{i=0}^{B-1} a_i} \operatorname{Re} \left(\sum_{i=0}^{B-1} a_i \langle \mathbf{u} | \mathbf{K}_i | \mathbf{u}_j \rangle \right) \right], \quad \zeta_1 = \frac{1}{2} \left[1 - \frac{1}{\sum_{i=0}^{B-1} a_i} \operatorname{Re} \left(\sum_{i=0}^{B-1} a_i \langle \mathbf{u} | \mathbf{K}_i | \mathbf{u}_j \rangle \right) \right]. \quad (59)$$

Therefore,

$$\text{Re} \left(\sum_{i=0}^{B-1} a_i \langle \mathbf{u} | \mathbf{K}_i | \mathbf{u}_j \rangle \right) = (2\zeta_0 - 1) \sum_{i=0}^{B-1} a_i = (1 - 2\zeta_1) \sum_{i=0}^{B-1} a_i. \quad (60)$$

The remaining blocks in Eq. (54) are treated similarly. If $B = M$, one circuit configuration estimates $\langle \mathbf{u} | \mathbf{K} | \mathbf{u}_j \rangle$. To estimate $\langle \mathbf{u} | \mathbf{K} | \mathbf{u} \rangle$, replace $\mathbf{V}_j$ in Fig. 7 with $\mathbf{V}$.

(b) a_i are general complex numbers The LCU coefficients a_i in Eq. (9) are real for the present problem, but for completeness we also consider complex coefficients. As in (a), amplitude-encode the coefficients $(a_0, \dots, a_{B-1})$ of the first block:

$$|\hat{\mathbf{a}}_1\rangle = \frac{1}{\sqrt{\sum_{i=0}^{B-1} |a_i|^2}} \sum_{i=0}^{B-1} a_i |i\rangle = \mathbf{A}_1 |0\rangle^{\otimes b}. \quad (61)$$

Figure 8 gives the circuit for general complex coefficients. Again, $\mathbf{V}_j$ prepares the normalized parameter-shifted state.

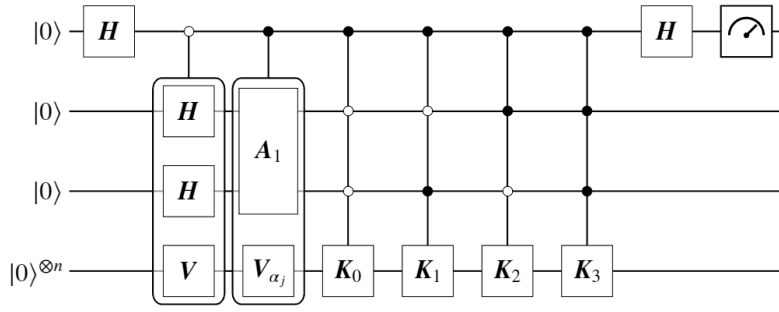


Figure 8 Block-Hadamard circuit for the weighted sum in the first block with general complex coefficients ($B = 4$)

The input state is $|\psi\rangle = |0\rangle|0\rangle^{\otimes b}|0\rangle^{\otimes n}$. After the first Hadamard gate, it becomes $|\psi\rangle = (|0\rangle + |1\rangle)|0\rangle^{\otimes b}|0\rangle^{\otimes n}/\sqrt{2}$. The next two layers are controlled. On the $|0\rangle$ branch of the first qubit, apply $\mathbf{H}^{\otimes b} \otimes \mathbf{V}(\alpha)$ to the target registers. On the $|1\rangle$ branch, apply $\mathbf{A}_1 \otimes \mathbf{V}_j$. The state then becomes

$$|\psi\rangle = \frac{1}{\sqrt{2}} \left(|0\rangle \mathbf{H}^{\otimes b} |0\rangle^{\otimes b} |\mathbf{u}\rangle + |1\rangle |\hat{\mathbf{a}}_1\rangle |\mathbf{u}_j\rangle \right). \quad (62)$$

After the quantum multiplexors, the state becomes

$$|\psi\rangle = \frac{1}{\sqrt{2}} \left[|0\rangle \frac{1}{(\sqrt{2})^b} \sum_{i=0}^{B-1} |i\rangle |\mathbf{u}\rangle + |1\rangle \frac{1}{\sqrt{\sum_{i=0}^{B-1} |a_i|^2}} \sum_{i=0}^{B-1} a_i |i\rangle \mathbf{K}_i |\mathbf{u}_j\rangle \right]. \quad (63)$$

The final Hadamard gate produces

$$\begin{aligned} |\psi\rangle = & \frac{1}{2} |0\rangle \left(\frac{1}{(\sqrt{2})^b} \sum_{i=0}^{B-1} |i\rangle |\mathbf{u}\rangle + \frac{1}{\sqrt{\sum_{i=0}^{B-1} |a_i|^2}} \sum_{i=0}^{B-1} a_i |i\rangle \mathbf{K}_i |\mathbf{u}_j\rangle \right) \\ & + \frac{1}{2} |1\rangle \left(\frac{1}{(\sqrt{2})^b} \sum_{i=0}^{B-1} |i\rangle |\mathbf{u}\rangle - \frac{1}{\sqrt{\sum_{i=0}^{B-1} |a_i|^2}} \sum_{i=0}^{B-1} a_i |i\rangle \mathbf{K}_i |\mathbf{u}_j\rangle \right). \end{aligned} \quad (64)$$

Measure the first qubit and denote the probabilities of $|0\rangle$ and $|1\rangle$ by ζ_0 and ζ_1 . Then

$$\begin{aligned}\zeta_0 &= \frac{1}{2} \left(1 + \frac{1}{\sqrt{B \sum_{i=0}^{B-1} |a_i|^2}} \operatorname{Re} \left(\sum_{i=0}^{B-1} a_i \langle \mathbf{u} | \mathbf{K}_i | \mathbf{u}_j \rangle \right) \right) \\ \zeta_1 &= \frac{1}{2} \left(1 - \frac{1}{\sqrt{B \sum_{i=0}^{B-1} |a_i|^2}} \operatorname{Re} \left(\sum_{i=0}^{B-1} a_i \langle \mathbf{u} | \mathbf{K}_i | \mathbf{u}_j \rangle \right) \right).\end{aligned}\quad (65)$$

Hence,

$$\operatorname{Re} \left(\sum_{i=0}^{B-1} a_i \langle \mathbf{u} | \mathbf{K}_i | \mathbf{u}_j \rangle \right) = (2\zeta_0 - 1) \sqrt{B \sum_{i=0}^{B-1} |a_i|^2} = (1 - 2\zeta_1) \sqrt{B \sum_{i=0}^{B-1} |a_i|^2}.\quad (66)$$

Other blocks use the same circuit structure. If $B = M$, one circuit configuration estimates $\langle \mathbf{u} | \mathbf{K} | \mathbf{u}_j \rangle$. For $\langle \mathbf{u} | \mathbf{K} | \mathbf{u} \rangle$, replace $\mathbf{V}_j$ in Fig. 7 with $\mathbf{V}$.

4.2 Block-Hadamard test for batched derivatives

Section 4.1 combined several unitary terms within one block, so that the potential energy and its derivative with respect to one parameter can each be estimated with one circuit configuration. With $B = M$, this nominally requires two configurations. A gradient-based optimizer, however, needs derivatives with respect to all α_j . Separate circuits for each parameter would still make the configuration count grow with N_α . We therefore encode the current state and each state shifted once by π in a state-index register. This enables batched estimation of all potential-energy derivatives.

Section 3.2 gave two ways to calculate the potential-energy gradient, in Eqs. (49) and (50): Eq. (49) requires batched estimates of $\langle \mathbf{u}(\boldsymbol{\alpha}) | \mathbf{K} | \mathbf{u}_{\alpha_j}(\boldsymbol{\alpha}) \rangle$ and $\langle \mathbf{f} | \mathbf{u}_{\alpha_j}(\boldsymbol{\alpha}) \rangle$ for $1 \leq j \leq N_\alpha$, whereas Eq. (50) requires $\langle \mathbf{u}(\boldsymbol{\alpha}_{j,\Delta\alpha}) | \mathbf{K} | \mathbf{u}(\boldsymbol{\alpha}_{j,\Delta\alpha}) \rangle$ and $\langle \mathbf{f} | \mathbf{u}(\boldsymbol{\alpha}_{j,\Delta\alpha}) \rangle$. To organize these calculations, note that

$$\begin{cases} \langle \mathbf{u}(\boldsymbol{\alpha}) | \mathbf{K} | \mathbf{u}_{\alpha_j}(\boldsymbol{\alpha}) \rangle = \langle 0 |^{\otimes n} \mathbf{V}^H(\boldsymbol{\alpha}) \mathbf{K} \mathbf{V}_{\alpha_j}(\boldsymbol{\alpha}) | 0 \rangle^{\otimes n} \\ \langle \mathbf{u}(\boldsymbol{\alpha}_{j,\Delta\alpha}) | \mathbf{K} | \mathbf{u}(\boldsymbol{\alpha}_{j,\Delta\alpha}) \rangle = \langle 0 |^{\otimes n} \mathbf{V}^H(\boldsymbol{\alpha}_{j,\Delta\alpha}) \mathbf{K} \mathbf{V}(\boldsymbol{\alpha}_{j,\Delta\alpha}) | 0 \rangle^{\otimes n} \\ \langle \mathbf{f} | \mathbf{u}_{\alpha_j}(\boldsymbol{\alpha}) \rangle = \langle 0 |^{\otimes n} \mathbf{F}^H \mathbf{V}_{\alpha_j} | 0 \rangle^{\otimes n}, \quad \langle \mathbf{f} | \mathbf{u}(\boldsymbol{\alpha}_{j,\Delta\alpha}) \rangle = \langle 0 |^{\otimes n} \mathbf{F}^H \mathbf{V}(\boldsymbol{\alpha}_{j,\Delta\alpha}) | 0 \rangle^{\otimes n} \end{cases}.\quad (67)$$

By Eq. (48), the derivative state differs from the normalized parameter-shifted state only by the classical factor $1/2$, so that batched evaluation of $\langle \mathbf{u}(\boldsymbol{\alpha}) | \mathbf{K} | \mathbf{u}_{\alpha_j}(\boldsymbol{\alpha}) \rangle$ reduces to batched evaluation of $\langle 0 |^{\otimes n} \mathbf{V}^H \mathbf{K} \mathbf{V}_j | 0 \rangle^{\otimes n}$. In addition, the finite-difference quadratic term reduces to $\langle 0 |^{\otimes n} \mathbf{V}_j^H \mathbf{K} \mathbf{V}_j | 0 \rangle^{\otimes n}$, and both overlaps with the load state are of the form $\langle 0 |^{\otimes n} \mathbf{F}^H \mathbf{V}_j | 0 \rangle^{\otimes n}$, up to the factor $1/2$ in the analytic derivative. We treat these three cases in turn.

(a) Batched evaluation of $\langle 0 |^{\otimes n} \mathbf{V}^H \mathbf{K} \mathbf{V}_j | 0 \rangle^{\otimes n}$ We need these overlaps for $\mathbf{V}_0, \mathbf{V}_1, \dots, \mathbf{V}_{N_\alpha-1}$. Decompose the stiffness matrix as $\mathbf{K} = \sum_{i=0}^{M-1} a_i \mathbf{K}_i$, so that the batched calculation becomes

$$\begin{aligned}\langle 0 |^{\otimes n} \mathbf{V}^H \mathbf{K} \mathbf{V}_j | 0 \rangle^{\otimes n} &= \sum_{i=0}^{M-1} a_i \langle 0 |^{\otimes n} \mathbf{V}^H \mathbf{K}_i \mathbf{V}_j | 0 \rangle^{\otimes n} \\ &= \sum_{i=0}^{B-1} a_i \langle 0 |^{\otimes n} \mathbf{V}^H \mathbf{K}_i \mathbf{V}_j | 0 \rangle^{\otimes n} + \sum_{i=B}^{2B-1} a_i \langle 0 |^{\otimes n} \mathbf{V}^H \mathbf{K}_i \mathbf{V}_j | 0 \rangle^{\otimes n} \\ &\quad + \dots + \sum_{i=M-B}^{M-1} a_i \langle 0 |^{\otimes n} \mathbf{V}^H \mathbf{K}_i \mathbf{V}_j | 0 \rangle^{\otimes n}.\end{aligned}\quad (68)$$

Partition $j \in \{0, 1, \dots, N_\alpha - 1\}$ into N_α/D groups of $D = 2^d$ indices. Without loss of generality, consider the first group:

$$\sum_{i=0}^{B-1} a_i \langle 0|^{\otimes n} \mathbf{V}^H \mathbf{K}_i \mathbf{V}_0 | 0 \rangle^{\otimes n}, \sum_{i=0}^{B-1} a_i \langle 0|^{\otimes n} \mathbf{V}^H \mathbf{K}_i \mathbf{V}_1 | 0 \rangle^{\otimes n}, \dots, \sum_{i=0}^{B-1} a_i \langle 0|^{\otimes n} \mathbf{V}^H \mathbf{K}_i \mathbf{V}_{D-1} | 0 \rangle^{\otimes n}. \quad (69)$$

The terms in Eq. (69) can be estimated with the circuit in Fig. 9, illustrated for $D = B = 4$. To simplify the quantum multiplexor, we assume nonnegative real a_i . For negative or complex coefficients, their phases can be absorbed into $\mathbf{K}_i$ as in Section 4.1(b).

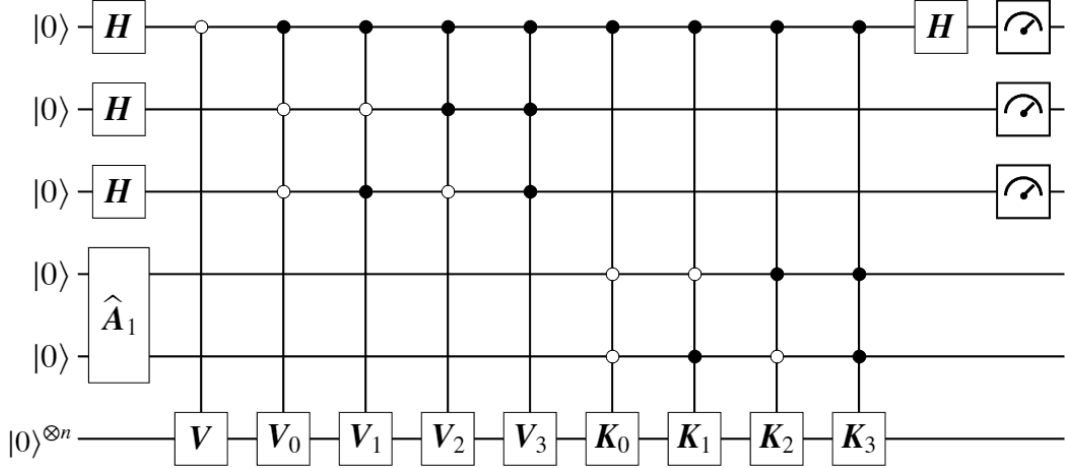


Figure 9 Cross-state block-Hadamard circuit for batched estimation of $\text{Re}\langle \mathbf{u}(\boldsymbol{\alpha}) | \mathbf{K} | \mathbf{u}_s \rangle$ ($D_K = B = 4$)

The input is $|\psi\rangle = |0\rangle|0\rangle^{\otimes d}|0\rangle^{\otimes b}|0\rangle^{\otimes n}$. After the circuit layers, the final state is

$$|\psi\rangle = \frac{1}{(\sqrt{2})^{d+2} \sqrt{\sum_{i=0}^{B-1} a_i}} \left(|0\rangle \sum_{j=0}^{D-1} |j\rangle \sum_{i=0}^{B-1} \sqrt{a_i} |i\rangle (|\mathbf{u}\rangle + \mathbf{K}_i \mathbf{V}_j |0\rangle^{\otimes n}) + |1\rangle \sum_{j=0}^{D-1} |j\rangle \sum_{i=0}^{B-1} \sqrt{a_i} |i\rangle (|\mathbf{u}\rangle - \mathbf{K}_i \mathbf{V}_j |0\rangle^{\otimes n}) \right). \quad (70)$$

Measure the first ancilla and the state-index register. Denote the joint probabilities of $|0\rangle|j\rangle$ and $|1\rangle|j\rangle$ by ζ_{0j} and ζ_{1j} , respectively. They are

$$\zeta_{0j} = \frac{1}{2D} \left[1 + \frac{\text{Re} \left(\sum_{i=0}^{B-1} a_i \langle 0|^{\otimes n} \mathbf{V}^H \mathbf{K}_i \mathbf{V}_j | 0 \rangle^{\otimes n} \right)}{\sum_{i=0}^{B-1} a_i} \right], \quad \zeta_{1j} = \frac{1}{2D} \left[1 - \frac{\text{Re} \left(\sum_{i=0}^{B-1} a_i \langle 0|^{\otimes n} \mathbf{V}^H \mathbf{K}_i \mathbf{V}_j | 0 \rangle^{\otimes n} \right)}{\sum_{i=0}^{B-1} a_i} \right]. \quad (71)$$

Therefore,

$$\begin{aligned} \text{Re} \left(\sum_{i=0}^{B-1} a_i \langle 0|^{\otimes n} \mathbf{V}^H \mathbf{K}_i \mathbf{V}_j | 0 \rangle^{\otimes n} \right) &= (1 - 2D\zeta_{1j}) \sum_{i=0}^{B-1} a_i = (2D\zeta_{0j} - 1) \sum_{i=0}^{B-1} a_i \\ &= D(\zeta_{0j} - \zeta_{1j}) \sum_{i=0}^{B-1} a_i, \quad 0 \leq j \leq D-1. \end{aligned} \quad (72)$$

The same circuit structure estimates all $\langle 0|^{\otimes n} \mathbf{V}^H \mathbf{K} \mathbf{V}_j | 0 \rangle^{\otimes n}$. If $B = M$ and $D = N_\alpha$, one circuit configuration suffices for all of them.

(b) Batched evaluation of $\langle 0|^{\otimes n} \mathbf{V}_j^H \mathbf{K} \mathbf{V}_j | 0 \rangle^{\otimes n}$ For the required quadratic terms, decompose $\mathbf{K}$ into its LCU terms. Then

$$\begin{aligned} \langle 0|^{\otimes n} \mathbf{V}_j^H \mathbf{K} \mathbf{V}_j | 0 \rangle^{\otimes n} &= \sum_{i=0}^{M-1} a_i \langle 0|^{\otimes n} \mathbf{V}_j^H \mathbf{K}_i \mathbf{V}_j | 0 \rangle^{\otimes n} \\ &= \sum_{i=0}^{B-1} a_i \langle 0|^{\otimes n} \mathbf{V}_j^H \mathbf{K}_i \mathbf{V}_j | 0 \rangle^{\otimes n} + \sum_{i=B}^{2B-1} a_i \langle 0|^{\otimes n} \mathbf{V}_j^H \mathbf{K}_i \mathbf{V}_j | 0 \rangle^{\otimes n} \\ &\quad + \cdots + \sum_{i=M-B}^{M-1} a_i \langle 0|^{\otimes n} \mathbf{V}_j^H \mathbf{K}_i \mathbf{V}_j | 0 \rangle^{\otimes n}. \end{aligned} \quad (73)$$

Again, partition $j \in \{0, 1, \dots, N_\alpha - 1\}$ into N_α/D groups of $D = 2^d$ indices and consider the first group. A circuit with the structure of Fig. 9 evaluates its terms. Replace state preparation $\mathbf{V}(\boldsymbol{\alpha})$ in the first branch by $\mathbf{V}_j$. The two branches then prepare $\mathbf{V}_j | 0 \rangle^{\otimes n}$ and $\mathbf{K}_i \mathbf{V}_j | 0 \rangle^{\otimes n}$. The final state is

$$\begin{aligned} |\psi\rangle &= \frac{1}{(\sqrt{2})^{d+2} \sqrt{\sum_{i=0}^{B-1} a_i}} \left[|0\rangle \sum_{j=0}^{D-1} |j\rangle \sum_{i=0}^{B-1} \sqrt{a_i} |i\rangle (\mathbf{V}_j | 0 \rangle^{\otimes n} + \mathbf{K}_i \mathbf{V}_j | 0 \rangle^{\otimes n}) \right. \\ &\quad \left. + |1\rangle \sum_{j=0}^{D-1} |j\rangle \sum_{i=0}^{B-1} \sqrt{a_i} |i\rangle (\mathbf{V}_j | 0 \rangle^{\otimes n} - \mathbf{K}_i \mathbf{V}_j | 0 \rangle^{\otimes n}) \right]. \end{aligned} \quad (74)$$

The joint probabilities ζ_{0j} and ζ_{1j} for $|0\rangle|j\rangle$ and $|1\rangle|j\rangle$ have the form of Eq. (71), with $\mathbf{V}^H$ replaced by $\mathbf{V}_j^H$. Accordingly, Eq. (72) becomes $\text{Re} \sum_{i=0}^{B-1} a_i \langle 0|^{\otimes n} \mathbf{V}_j^H \mathbf{K}_i \mathbf{V}_j | 0 \rangle^{\otimes n} = D(\zeta_{0j} - \zeta_{1j}) \sum_{i=0}^{B-1} a_i$, for $0 \leq j \leq D-1$. If $B = M$ and $D = N_\alpha$, one circuit configuration suffices for all such quadratic terms.

(c) Batched evaluation of $\langle 0|^{\otimes n} \mathbf{F}^H \mathbf{V}_j | 0 \rangle^{\otimes n}$ We need an overlap of this form for $\mathbf{V}_0, \mathbf{V}_1, \dots, \mathbf{V}_{N_\alpha-1}$. No stiffness matrix is involved, so a unitary-term index register is unnecessary. Partition $j \in \{0, 1, \dots, N_\alpha - 1\}$ into N_α/D groups of $D = 2^d$ indices. For the first group,

$$\langle 0|^{\otimes n} \mathbf{F}^H \mathbf{V}_0 | 0 \rangle^{\otimes n}, \quad \langle 0|^{\otimes n} \mathbf{F}^H \mathbf{V}_1 | 0 \rangle^{\otimes n}, \quad \dots, \quad \langle 0|^{\otimes n} \mathbf{F}^H \mathbf{V}_{D-1} | 0 \rangle^{\otimes n}. \quad (75)$$

The terms in Eq. (75) can be estimated by the circuit in Fig. 10, shown for $D = 4$.

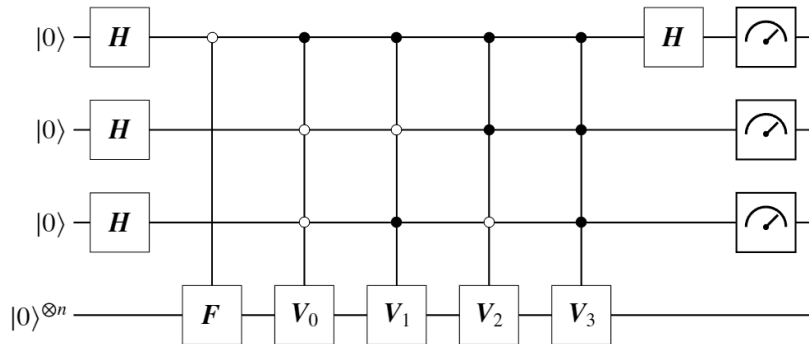


Figure 10 Block-Hadamard circuit for batched estimation of $\text{Re} \langle 0|^{\otimes n} \mathbf{F}^H \mathbf{V}_s | 0 \rangle^{\otimes n}$ ($D_f = 4$)

The input is $|\psi\rangle = |0\rangle|0\rangle^{\otimes d}|0\rangle^{\otimes n}$. After the circuit layers, the final state is

$$|\psi\rangle = \frac{1}{(\sqrt{2})^{d+2}} \left(|0\rangle \sum_{j=0}^{D-1} |j\rangle (\mathbf{F} + \mathbf{V}_j) | 0 \rangle^{\otimes n} + |1\rangle \sum_{j=0}^{D-1} |j\rangle (\mathbf{F} - \mathbf{V}_j) | 0 \rangle^{\otimes n} \right). \quad (76)$$

Measure the first ancilla and the state-index register. Denote the joint probabilities of $|0\rangle|j\rangle$ and $|1\rangle|j\rangle$ by ζ_{0j} and ζ_{1j} , respectively. They are

$$\zeta_{0j} = \frac{1}{2D} [1 + \text{Re}(\langle 0|^{\otimes n} \mathbf{F}^H \mathbf{V}_j | 0 \rangle^{\otimes n})], \quad \zeta_{1j} = \frac{1}{2D} [1 - \text{Re}(\langle 0|^{\otimes n} \mathbf{F}^H \mathbf{V}_j | 0 \rangle^{\otimes n})]. \quad (77)$$

Hence,

$$\begin{aligned} \text{Re}(\langle 0|^{\otimes n} \mathbf{F}^H \mathbf{V}_j | 0 \rangle^{\otimes n}) &= 1 - 2D\zeta_{1j} = 2D\zeta_{0j} - 1 \\ &= D(\zeta_{0j} - \zeta_{1j}), \quad 0 \leq j \leq D-1. \end{aligned} \quad (78)$$

The same circuit structure estimates all $\langle 0|^{\otimes n} \mathbf{F}^H \mathbf{V}_j | 0 \rangle^{\otimes n}$. If $D = N_\alpha$, one circuit configuration suffices, so that the total potential energy and all its parameter derivatives can be estimated in batches with a constant number of circuit configurations. Define

$$A_s^{(K)} = \text{Re}(\langle 0|^{\otimes n} \mathbf{V}^H \mathbf{K} \mathbf{V}_s | 0 \rangle^{\otimes n}), \quad L_s^{(f)} = \text{Re}(\langle 0|^{\otimes n} \mathbf{F}^H \mathbf{V}_s | 0 \rangle^{\otimes n}), \quad 0 \leq s \leq N_\alpha. \quad (79)$$

Here, $A_s^{(K)}$ is recovered from the batched result in Eq. (72). For the finite-difference method in Eq. (50), use the corresponding diagonal term in Eq. (73). The quantity $L_s^{(f)}$ is recovered from Eq. (78), and the potential-energy gradient is then reconstructed as

$$\frac{\partial \Pi}{\partial c} = cA_0^{(K)} - \|\mathbf{f}\|_2 L_0^{(f)}, \quad \frac{\partial \Pi}{\partial \alpha_j} = \frac{1}{2} [c^2 A_j^{(K)} - c \|\mathbf{f}\|_2 L_j^{(f)}], \quad 1 \leq j \leq N_\alpha. \quad (80)$$

This reduction concerns the number of distinct circuit configurations: each configuration still requires repeated sampling, and the joint probability for an individual state index decreases as $1/D$. The shots required for a specified statistical precision therefore grow with the index-space size, variance, and target confidence level.

Remark 3 The batched method relies on quantum multiplexors implementing $\bigoplus_{j=0}^{D-1} \mathbf{V}_j$ and $\bigoplus_{i=0}^{B-1} \mathbf{K}_i$. Their realization is nontrivial and generally uses controlled gates and uniformly controlled rotation decompositions [46]. Noise in current hardware may limit their implementation. Practical deployment of the algorithm thus depends on hardware progress. Nevertheless, the LCU decomposition in Section 3.1 makes the $\mathbf{K}_i$ multiplexor primarily a combination of cyclic permutations, R_y gates, and diagonal unitaries. If the ansatz in Fig. 1 is used, the $\mathbf{V}_j$ multiplexor also mainly comprises R_y gates. Remark 2 and Appendix 2 give the assumptions under which these gates can be implemented in $O(\text{poly}(\log N))$ time.

Remark 4 The block-Hadamard test presented here batches overlaps of the form $\langle \mathbf{u} | \mathbf{K}_i | \mathbf{u} \rangle$, and the same idea also applies to $\langle \mathbf{u} | \mathbf{K}_i^H \mathbf{K}_j | \mathbf{u} \rangle$ in conventional loss functions. Appendix 3 gives the corresponding quantum circuit.

5. Numerical examples

This section presents three proof-of-concept examples: a two-dimensional truss, a plane-stress plate, and a three-dimensional steady-state heat-conduction problem. The first two test static structural problems, and the third tests a three-dimensional system with one degree of freedom per node. Together, they assess the voxel-grid LCU decomposition and minimum-potential-energy formulation from Section 3. All results were obtained with classical numerical programs in MATLAB.

Let $\mathbf{u}_{\text{ref}}$ denote the displacement or nodal-temperature vector from classical finite-element analysis, and let $\mathbf{u}$ denote the corresponding field reconstructed from the ansatz. The relative error is

$$\varepsilon = \frac{\|\mathbf{u} - \mathbf{u}_{\text{ref}}\|_1}{\|\mathbf{u}_{\text{ref}}\|_1}. \quad (81)$$

Voxel-BVQLS uses the minimum-potential-energy objective in Eq. (44) and the voxel-grid LCU decomposition. For linear static problems, this objective is the total potential energy. The comparison methods VQLS-G, VQLS-L, and VQLS-R use the global, local, and residual losses from Section 2.3.2, respectively. Pauli-VQLS denotes the comparison method based on a Pauli decomposition.

Remark 5 The proposed block-Hadamard circuits estimate the total potential energy and its derivatives in batches. Under ideal noiseless conditions, different circuits estimating the same physical quantity yield the same result. The present examples

therefore do not quantify the computational-efficiency gain of block-Hadamard tests over conventional Hadamard tests. We provide only a qualitative assessment. As shown in Section 4, block-Hadamard tests reduce the number of circuit configurations needed for the total potential energy and its derivatives. If the block size equals the number of unitary terms, two types of circuit configuration are sufficient in principle.

Remark 6 The advantage of the proposed voxel-grid LCU decomposition is measured by its number of unitary terms. In each example, we decompose the stiffness matrix both by this method and by a Pauli decomposition, then compare the numbers of nonzero unitary terms.

Remark 7 The minimum-potential-energy objective in Eq. (44) is strictly convex over the full physical-field space, but its composition with a parameterized ansatz is generally nonconvex. The following comparisons describe numerical behavior for the specified ansatz, initial values, and stopping criteria and do not provide a general convergence guarantee for any objective or optimizer.

5.1 Truss problem

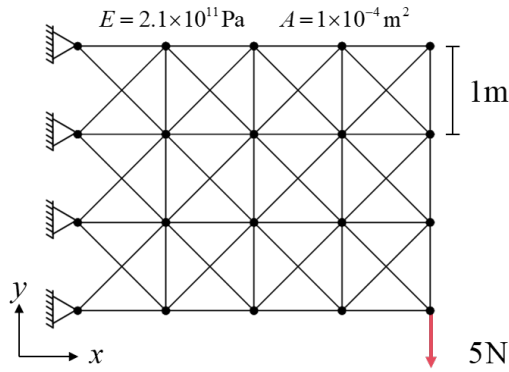


Figure 11 Geometry, boundary conditions, and applied load of the two-dimensional truss

The first example is the two-dimensional truss in Fig. 11. It has 20 nodes, two degrees of freedom per node, and a nodal spacing of 1 m. The bars have cross-sectional area $A = 10^{-4} \text{ m}^2$ and Young's modulus $E = 2.1 \times 10^{11} \text{ Pa}$. The nodes on the left edge are fixed. A concentrated force of 5 N acts downward at the lower-right node. After imposing the constraints, $N = 32$ degrees of freedom remain. This is a two-dimensional problem with two degrees of freedom per node. Table 3 shows 51 nonzero unitary terms from the Pauli decomposition and 18 from the proposed voxel-grid LCU decomposition, a reduction of about 65%. We solve the problem with Voxel-BVQLS, VQLS-G, VQLS-L, and VQLS-R. All four methods use the ansatz in Fig. 12, with every initial parameter set to 1 and with 3, 5, or 7 layers. Sequential quadratic programming (SQP) is used with at most 500 iterations. The allowed difference between successive parameter iterates (StepTolerance) is 10^{-10} . Figure 13 plots relative error against iteration count for the four methods and three ansatz depths.

Table 3 Nonzero unitary terms in Pauli and voxel-grid decompositions of the truss stiffness matrix

Decomposition	Pauli LCU	Voxel-grid LCU
Number of nonzero unitary terms	51	18

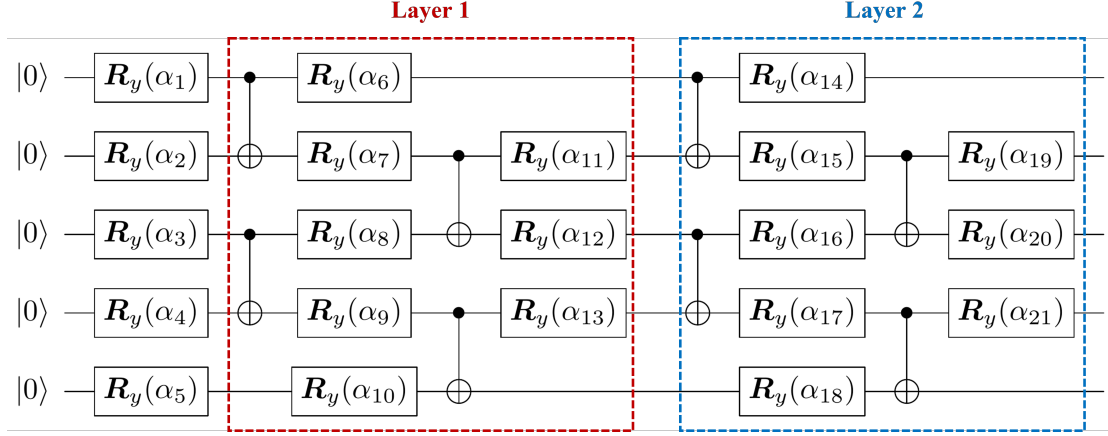


Figure 12 Ansatz circuit for the two-dimensional truss example

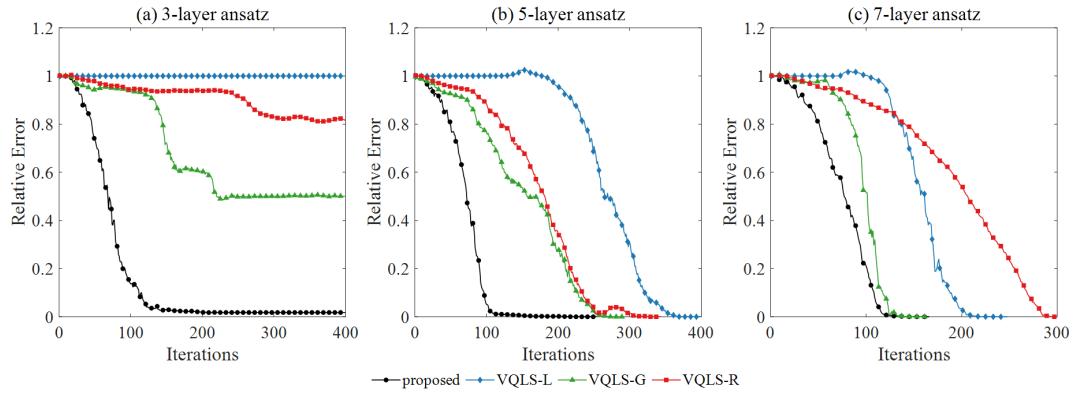


Figure 13 Displacement L_1 relative-error curves for four methods with 3-, 5-, and 7-layer ansatz circuits in the two-dimensional truss example

Figure 13 shows that, with the same initial values and optimization settings, Voxel-BVQLS reduces the relative error rapidly and stabilizes after about 100–150 iterations at all three ansatz depths. With five layers, it reaches low error after about 110 iterations, whereas VQLS-G, VQLS-R, and VQLS-L need about 250, 320, and 370 iterations, respectively, to approach zero error. With three layers, none of those three comparison methods finds a low-error solution within the plotted range. With seven layers, VQLS-G also reduces the error relatively quickly, while VQLS-L and VQLS-R require more iterations. Under the ansatz, initialization, and SQP settings of this example, these results indicate that the minimum-potential-energy objective improves optimization and alleviates the high-error stagnation associated here with barren plateaus at shallow depth.

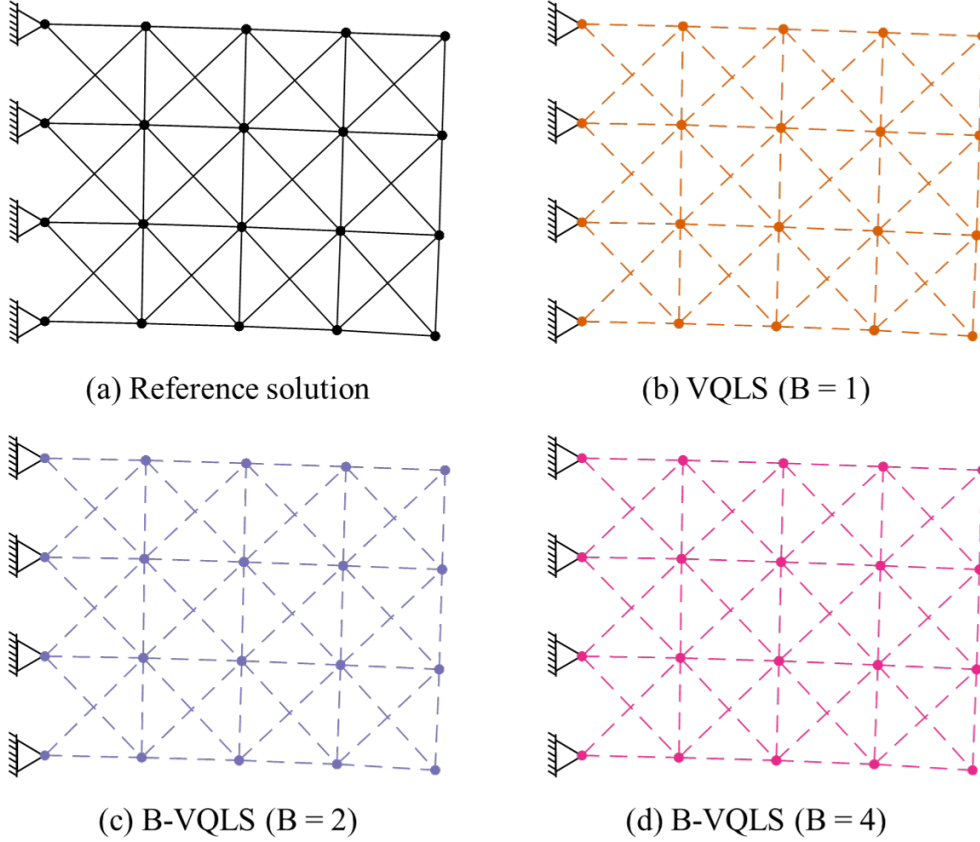


Figure 14 Reference and computed deformed truss configurations for block sizes $B = 1, 2$, and 4

Figure 14 compares the truss-node positions from the classical finite element reference, the unblocked scheme ($B = 1$), and two block-Hadamard schemes ($B = 2, 4$). All three schemes reproduce the overall deformation under load. At the plotted scale, the loaded and interior nodes are close to the reference positions. The $B = 1$, $B = 2$, and $B = 4$ results coincide with the finite element reference in the figure. Thus, changing the unitary-term block size does not visibly degrade nodal-displacement accuracy in this noiseless example. Blocking changes how the overlaps are organized and estimated. With exact summation, the stiffness matrix and variational objective are unchanged. The block-Hadamard test therefore combines several term-wise overlap tests into fewer circuit configurations while retaining the plotted nodal-displacement accuracy in this example.

5.2 Plane-stress problem

The second example is the plane-stress plate in Fig. 15. It has 36 nodes with two degrees of freedom per node. Its length, width, and thickness are 7 mm, 3 mm, and 1 mm, respectively. Materials A and B have Young’s moduli of 2.1×10^5 MPa and 1.1×10^5 MPa, respectively, and both have a Poisson’s ratio of 0.3. The left-edge nodes are fixed. A 10N force acts in the positive x direction at the lower-right node, and a 15N force acts in the positive y direction at the upper-right node. After imposing the constraints, $N = 64$ degrees of freedom remain. This is a two-dimensional problem with two degrees of freedom per node. Table 4 lists 224 nonzero unitary terms from the Pauli decomposition and 18 from the proposed voxel-grid LCU

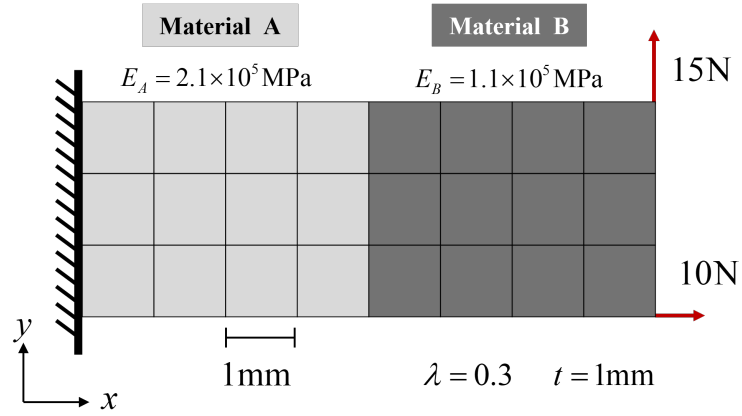


Figure 15 Geometry, material layout, boundary conditions, and applied loads of the plane-stress model

decomposition, a reduction of about 92%. The four methods use the ansatz in Fig. 16, with every initial parameter set to 1 and depths of 4, 7, and 10 layers. SQP is run for at most 3000 iterations. To make every curve in Fig. 17 cover the same iteration range, the early-stopping tolerances for function value, step size, and first-order optimality are all set to zero. Figure 17 compares relative-error curves across methods and depths. Figure 18 shows spatial displacement-field errors for Voxel-BVQLS at the same depths.

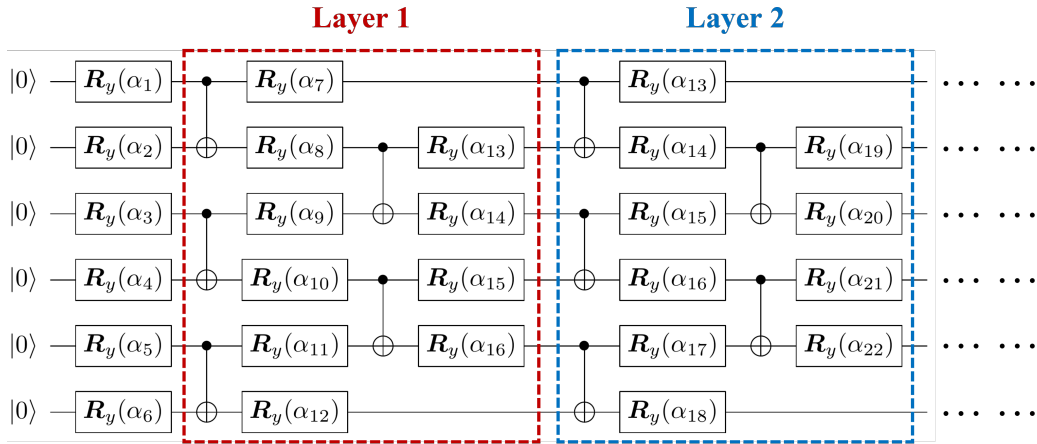


Figure 16 Ansatz circuit for the plane-stress example

Table 4 Nonzero unitary terms in Pauli and voxel-grid decompositions of the plane-stress stiffness matrix

Decomposition	Pauli LCU	Voxel-grid LCU
Number of nonzero unitary terms	224	18

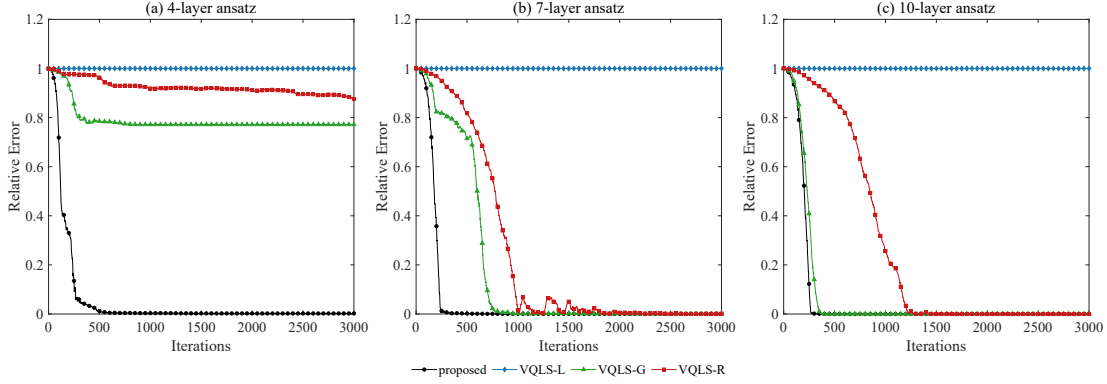


Figure 17 Displacement L_1 relative-error curves for four methods with 4-, 7-, and 10-layer ansatz circuits in the plane-stress example

Figure 17 shows that, with the same initial values and a 3000-iteration budget, Voxel-BVQLS reaches a low displacement-error range earlier at all three depths. With four layers, its final error is about 2.15×10^{-3} . None of the other methods reaches 10^{-2} in the plotted range. With seven layers, VQLS-G and VQLS-R reach that threshold after about 780 and 1000 iterations, respectively, later than Voxel-BVQLS. The VQLS-L error remains near 1. With ten layers, VQLS-G reaches 10^{-2} after about 350 iterations, whereas VQLS-R needs about 1220. Under the ansatz, initialization, and SQP settings used here, the minimum-potential-energy objective reaches a 1% displacement relative error earlier at each depth and alleviates the high-error stagnation seen at shallow depth.

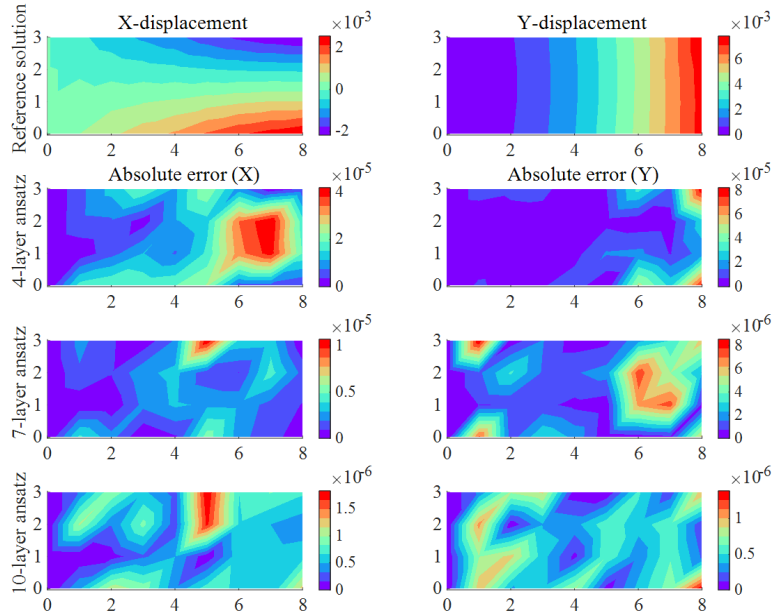


Figure 18 Reference x - and y -displacement fields and absolute errors for 4-, 7-, and 10-layer ansatz circuits in the plane-stress example

The upper row of Fig. 18 shows the classical finite element reference displacements in the x and y directions, with values on the order of 10^{-3} . The other rows show absolute errors for Voxel-BVQLS with 4, 7, and 10 layers. At four layers, the upper color-scale limits for the x - and y -direction errors are about 4×10^{-5} and 8×10^{-5} . The larger errors occur mainly on the right

side of the plate. As depth increases, both maximum color-scale values decline. At ten layers, they are about 1.5×10^{-6} and 10^{-6} . Under the tested depths and optimization settings, the method reproduces the overall reference displacement distribution, and deeper ansatz circuits reduce local error. These are field approximations for the tested problem, not a general accuracy guarantee.

5.3 Three-dimensional heat-conduction problem

The third example is the three-dimensional steady-state heat-conduction model in Fig. 19. It contains $17 \times 7 \times 3$ eight-node voxel elements and $18 \times 8 \times 4 = 576$ nodes, each with one temperature degree of freedom. The two materials have thermal conductivities $k_1 = 2.33 \text{ W cm}^{-1} \text{ }^\circ\text{C}^{-1}$ and $k_2 = 0.33 \text{ W cm}^{-1} \text{ }^\circ\text{C}^{-1}$. A uniform heat flux of $q = 2.5 \text{ W cm}^{-2}$ is applied to the top surface. The left and right end faces are held at 0°C . After applying these boundary conditions, $N = 512$ temperature degrees of freedom remain. This is a three-dimensional problem with one degree of freedom per node. Table 5 gives 228 nonzero unitary terms for the Pauli decomposition and 18 for the voxel-grid LCU decomposition, a reduction of about 92%. All four methods use the ansatz in Fig. 20, with initial parameters equal to 1 and depths of 4, 10, and 16 layers. SQP is run for at most 500 iterations with StepTolerance 10^{-10} .

Figure 21 shows that the Voxel-BVQLS relative error starts to decrease markedly earlier at all three depths. With four layers, it falls below 10^{-1} after about 120 iterations and stabilizes near 4×10^{-2} . With ten and sixteen layers, it reaches a lower level after about 200 and 250 iterations, respectively, and continues to decrease. By comparison, VQLS-L stays near an error of 1 at all depths, and VQLS-R finds no low-error solution within 500 iterations. VQLS-G retains a noticeable residual error at four and ten layers. At sixteen layers, it also reduces the error close to zero, but requires more iterations than Voxel-BVQLS. For this three-dimensional example and the stated initialization and SQP settings, the minimum-potential-energy objective alleviates high-error stagnation. Greater ansatz depth also improves the solution accuracy.

The classical finite element reference in Fig. 22 shows temperature rising from the two 0°C end faces toward the interior, with a hot region near the heated surface and the center. At four layers, the absolute temperature error of Voxel-BVQLS has an upper color-scale limit of about 0.8°C . This falls to about 0.15°C at ten layers and 0.04°C at sixteen layers. The spatial decline in error with depth is consistent with the relative-error trends in Fig. 21. Thus, the solutions approach the reference both in overall vector norm and in the temperature-field pattern.

Table 5 Nonzero unitary terms in Pauli and voxel-grid decompositions of the heat-conduction stiffness matrix

Decomposition	Pauli LCU	Voxel-grid LCU
Number of nonzero unitary terms	228	18

These three noiseless examples show fewer nonzero unitary terms from the voxel-grid LCU decomposition than from the Pauli decomposition. Optimization of the minimum-potential-energy objective also yields approximations to the displacement and temperature fields.

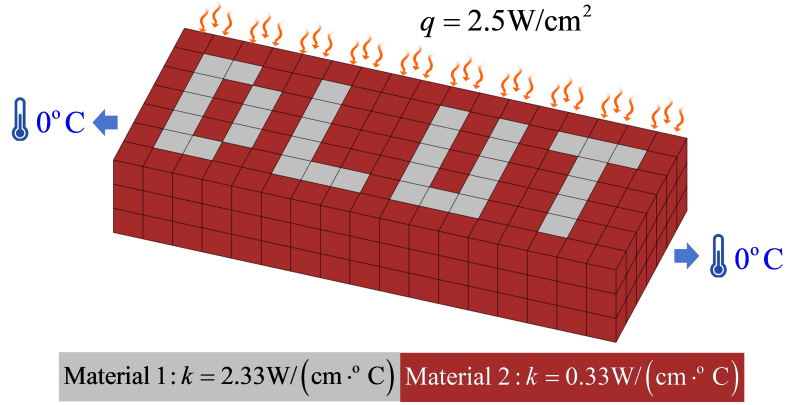


Figure 19 Voxel geometry, material distribution, applied heat flux, and temperature boundary conditions of the three-dimensional heat-conduction model

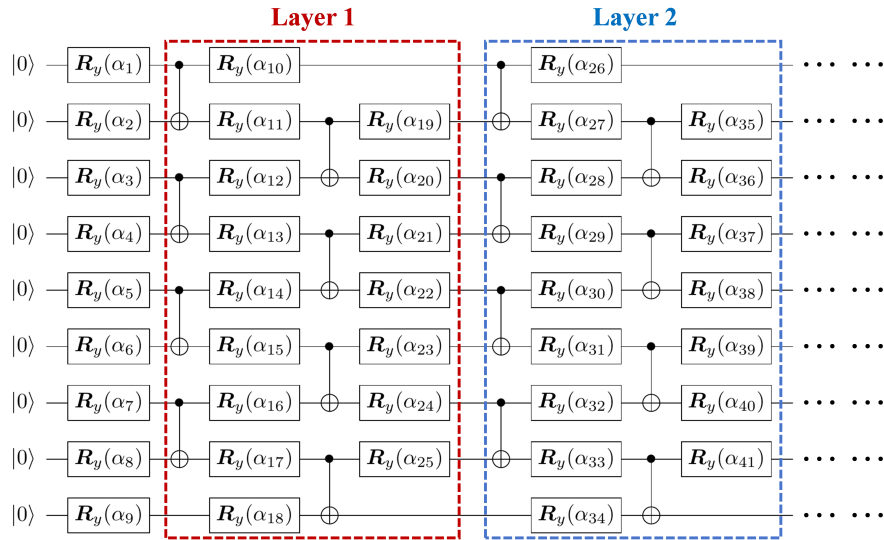


Figure 20 Ansatz circuit for the three-dimensional heat-conduction example

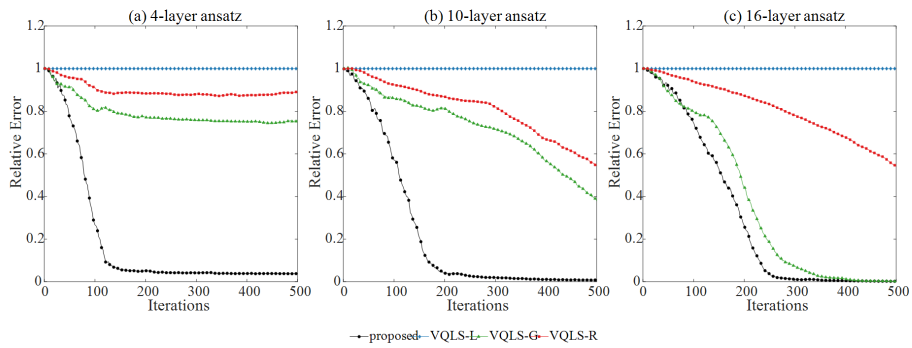


Figure 21 Temperature L_1 relative-error curves for four methods with 4-, 10-, and 16-layer ansatz circuits in the three-dimensional heat-conduction example

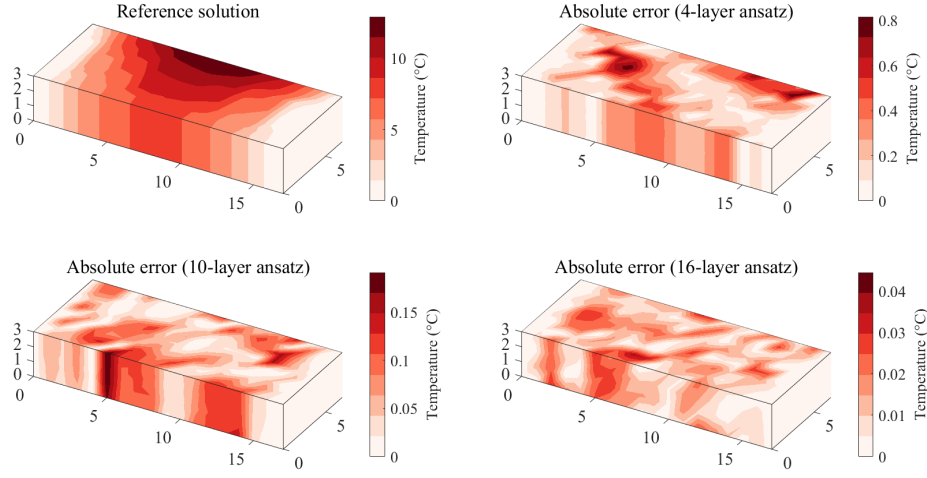


Figure 22 Reference temperature field and absolute errors at different ansatz depths for the three-dimensional heat-conduction problem

6. Conclusions

To address large sparse linear systems in static solid mechanics, we developed a voxel-based block variational quantum linear solver (Voxel-BVQLS) on regular voxel grids. It combines structured matrix decomposition, the principle of minimum potential energy, and batched quantum tests in a hybrid quantum–classical finite element workflow. First, for block-banded voxel-grid stiffness matrices, we use cyclic permutation matrices and block-diagonal matrices to construct an LCU decomposition whose upper bound on the number of unitary terms does not grow with the number of mesh nodes. Second, using the linear-static total potential energy as the variational objective mitigates barren plateaus in the studied settings. Third, we propose a block-Hadamard test that directly estimates weighted sums of multiple inner products, reducing the number of circuit configurations required per iteration.

Three numerical examples assessed the method. In the two-dimensional truss, plane-stress, and three-dimensional steady-state heat-conduction problems, the voxel-grid LCU decomposition has 18 nonzero unitary terms in each case, compared with 51, 224, and 228 Pauli terms, respectively. The computed displacement and temperature fields also approximate the reference solutions with the errors reported above. Within these examples, the method reduces the numbers of unitary terms and circuit configurations and attains the reported field errors in fewer optimization iterations than the comparison VQLS methods.

Several limitations remain. All calculations were performed on a classical computer. Finite-shot sampling, quantum gate noise, and measurement noise were not included. The block-Hadamard test relies on quantum multiplexors, whose gate-level cost and compatibility with current hardware require separate assessment. Future work will add gate-level resource estimates and noisy simulation, and develop improved optimizers for engineering applications.

Appendix 1. Gate representation of the singular value decomposition of a real 2×2 matrix (proof of Lemma 1)

Lemma 1. The singular value decomposition of any real 2×2 matrix $\mathbf{A} = \begin{bmatrix} a & b \\ c & d \end{bmatrix}$ can be written as

$$\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T, \quad \mathbf{U} = \mathbf{R}_y(2\gamma), \quad \mathbf{V} = \mathbf{R}_y(2\theta), \quad \mathbf{\Sigma} = \text{diag}(\sigma_1, \sigma_2), \quad (82)$$

where $\sigma_1 \geq \sigma_2 \geq 0$, and $\mathbf{R}_y(\varphi)$ is defined in Table 1: $\mathbf{R}_y(\varphi) = \begin{bmatrix} \cos(\varphi/2) & -\sin(\varphi/2) \\ \sin(\varphi/2) & \cos(\varphi/2) \end{bmatrix}$. The rotation angles 2θ and 2γ are given by Eqs. (93) and (94). If $\det \mathbf{A} \geq 0$, both $\mathbf{U}$ and $\mathbf{V}$ can have the pure-rotation form of Eq. (82). If $\det \mathbf{A} < 0$, exactly one

of the two factors requires an additional $\mathbf{Z}$ gate as in Eq. (95). The proof follows.

The key step is an orthogonal diagonalization of the real symmetric positive-semidefinite matrix $\mathbf{A}^T \mathbf{A}$. First, expand it as

$$\mathbf{A}^T \mathbf{A} = \begin{bmatrix} a & c \\ b & d \end{bmatrix} \begin{bmatrix} a & b \\ c & d \end{bmatrix} = \begin{bmatrix} a^2 + c^2 & ab + cd \\ ab + cd & b^2 + d^2 \end{bmatrix}. \quad (83)$$

Let $T = \text{tr}(\mathbf{A}^T \mathbf{A}) = a^2 + b^2 + c^2 + d^2$ and $D = \det(\mathbf{A}^T \mathbf{A}) = (\det \mathbf{A})^2 = (ad - bc)^2$. The discriminant of the characteristic equation is

$$\Delta = \sqrt{T^2 - 4D} = \sqrt{(a^2 + c^2 - b^2 - d^2)^2 + 4(ab + cd)^2} \geq 0. \quad (84)$$

This quantity is nonnegative, ensuring that the eigenvalues of $\mathbf{A}^T \mathbf{A}$ are real and nonnegative. Solving $\lambda^2 - T\lambda + D = 0$ gives

$$\lambda_1 = \frac{T + \Delta}{2}, \quad \lambda_2 = \frac{T - \Delta}{2}. \quad (85)$$

The singular values are the nonnegative square roots of the eigenvalues. In descending order, they form

$$\boldsymbol{\Sigma} = \begin{bmatrix} \sigma_1 & \\ & \sigma_2 \end{bmatrix}, \quad \sigma_1 = \sqrt{\lambda_1} = \sqrt{\frac{T + \Delta}{2}}, \quad \sigma_2 = \sqrt{\lambda_2} = \sqrt{\frac{T - \Delta}{2}}. \quad (86)$$

The columns of the matrix of right singular vectors $\mathbf{V}$ are orthonormal eigenvectors of $\mathbf{A}^T \mathbf{A}$ satisfying

$$(\mathbf{A}^T \mathbf{A} - \lambda_i \mathbf{I}_2) \mathbf{v}_i = \mathbf{0}, \quad i = 1, 2. \quad (87)$$

For the largest eigenvalue λ_1 , substitute Eq. (83) into Eq. (87). A normalized eigenvector satisfying the result is

$$\mathbf{v}_1 = \frac{1}{N_1} \begin{bmatrix} ab + cd \\ \lambda_1 - a^2 - c^2 \end{bmatrix}, \quad N_1 = \sqrt{(ab + cd)^2 + (\lambda_1 - a^2 - c^2)^2}. \quad (88)$$

By orthogonality of eigenvectors of a real symmetric matrix, an eigenvector for λ_2 can be chosen as $\mathbf{v}_2 = (-v_{12}, v_{11})^T$, where v_{1i} is component i of $\mathbf{v}_1$. Thus, the matrix of right singular vectors is

$$\mathbf{V} = [\mathbf{v}_1 \ \mathbf{v}_2] = \frac{1}{N_1} \begin{bmatrix} ab + cd & -(\lambda_1 - a^2 - c^2) \\ \lambda_1 - a^2 - c^2 & ab + cd \end{bmatrix}. \quad (89)$$

The matrix of left singular vectors follows from $\mathbf{A}\mathbf{V} = \mathbf{U}\boldsymbol{\Sigma}$: if $\sigma_2 > 0$, then $\mathbf{A}$ has full rank, $\mathbf{U} = \mathbf{A}\mathbf{V}\boldsymbol{\Sigma}^{-1}$, and hence $\mathbf{u}_i = \mathbf{A}\mathbf{v}_i/\sigma_i$ for $i = 1, 2$, giving

$$\mathbf{U} = [\mathbf{u}_1 \ \mathbf{u}_2] = \frac{1}{\sigma_1 \sigma_2} \begin{bmatrix} (av_{11} + bv_{12}) \sigma_2 & (av_{21} + bv_{22}) \sigma_1 \\ (cv_{11} + dv_{12}) \sigma_2 & (cv_{21} + dv_{22}) \sigma_1 \end{bmatrix}. \quad (90)$$

If $\sigma_2 = 0$, $\mathbf{u}_1 = \mathbf{A}\mathbf{v}_1/\sigma_1$ still determines the first vector. Choose $\mathbf{u}_2$ as a unit vector orthogonal to $\mathbf{u}_1$.

We now express the orthogonal factors $\mathbf{U}$ and $\mathbf{V}$ as rotation gates directly implementable in a quantum circuit. Since $\mathbf{A}^T \mathbf{A} = \mathbf{V} \text{diag}(\lambda_1, \lambda_2) \mathbf{V}^T$, choose $\mathbf{V}$ as the rotation matrix

$$\mathbf{V} = \mathbf{R}_y(2\theta) = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}. \quad (91)$$

Expand $\mathbf{V} \text{diag}(\lambda_1, \lambda_2) \mathbf{V}^T$ and equate its entries to those in Eq. (83). This yields

$$ab + cd = \frac{\lambda_1 - \lambda_2}{2} \sin 2\theta, \quad (a^2 + c^2) - (b^2 + d^2) = (\lambda_1 - \lambda_2) \cos 2\theta. \quad (92)$$

Dividing the two equations eliminates $\lambda_1 - \lambda_2$ and gives an explicit constraint on the rotation angle: $\tan 2\theta = \frac{2(ab + cd)}{(a^2 + c^2) - (b^2 + d^2)}$. To retain quadrant information when $a^2 + c^2 = b^2 + d^2$, compute the angle with the two-argument arctangent:

$$2\theta = \text{atan2}(2(ab + cd), a^2 + c^2 - b^2 - d^2). \quad (93)$$

Similarly, substitute Eq. (89) into $\mathbf{U} = \mathbf{A}\mathbf{V}\mathbf{\Sigma}^{-1}$. Setting $\mathbf{U} = \mathbf{R}_y(2\gamma)$ and comparing matrix entries gives the left rotation angle:

$$\gamma = \text{atan2}(c \cos \theta + d \sin \theta, a \cos \theta + b \sin \theta). \quad (94)$$

In the degenerate case $\lambda_1 = \lambda_2 > 0$, we have $\mathbf{A}^T \mathbf{A} = \lambda_1 \mathbf{I}_2$ and $\tan 2\theta$ is undefined, so one may choose $\theta = 0$, $\mathbf{V} = \mathbf{I}_2 = \mathbf{R}_y(0)$, and $\mathbf{U} = \mathbf{A}/\sigma_1$. If $\mathbf{A} = \mathbf{0}$, choose $\mathbf{U} = \mathbf{V} = \mathbf{I}_2$. The rotation angles are not unique in these cases, but the decomposition remains valid.

Finally, consider the determinant signs. Every real orthogonal 2×2 matrix $\mathbf{Q}$ can be written as

$$\mathbf{Q} = \mathbf{R}_y(2\eta) \mathbf{Z}^s, \quad s = \frac{1 - \det(\mathbf{Q})}{2} \in \{0, 1\}, \quad (95)$$

where $\mathbf{Z} = \text{diag}(1, -1)$. If $\det \mathbf{A} \geq 0$ and $\mathbf{A}$ has full rank, one can consistently choose $\det \mathbf{U} = \det \mathbf{V} = 1$, so both factors in Eq. (82) are pure rotations. If $\det \mathbf{A} < 0$, exactly one factor contains a reflection and requires a $\mathbf{Z}$ gate as in Eq. (95). Thus, regardless of the determinant sign, every 2×2 orthogonal block $\mathbf{U}_n$ or $\mathbf{V}_n$ in Eqs. (34)–(38) can be implemented with a single-qubit R_y rotation and at most one $\mathbf{Z}$ gate. This completes the proof.

Appendix 2. Quantum circuit for a diagonal phase unitary

Let $N = 2^n$ and consider the diagonal unitary

$$\mathbf{D}_\theta = \bigoplus_{i=0}^{N-1} e^{i\theta_i}, \quad \mathbf{D}_\theta |\mathbf{y}\rangle = \sum_{i=0}^{N-1} e^{i\theta_i y_i} |i\rangle, \quad (96)$$

where $|\mathbf{y}\rangle = \sum_i y_i |i\rangle$. By the definition of a quantum multiplexor, Eq. (96) is a uniformly controlled phase gate on n qubits. Suppose the N phases take only N_s distinct values, denoted by $\{\hat{\theta}_1, \dots, \hat{\theta}_{N_s}\}$. Let $g(i) \in \{1, \dots, N_s\}$ identify the class of θ_i , so that $\theta_i = \hat{\theta}_{g(i)}$. Equation (96) can then be implemented with N_s ancilla qubits. First, encode the class in the ancilla register using one-hot encoding. Next, apply rotations to the ancillas in parallel. Finally, uncompute the register.

A reversible lookup oracle O_g writes the class information. It is defined by

$$O_g : |i\rangle |0\rangle^{\otimes N_s} \mapsto |i\rangle |m_{g(i)}\rangle. \quad (97)$$

The state $|m_j\rangle$ is a one-hot state on N_s qubits: qubit j is $|1\rangle$ and all others are $|0\rangle$. Each $|m_j\rangle$ corresponds to an angle $\hat{\theta}_j$.

Let the input at the left of the circuit be $|\psi\rangle = (\sum_i y_i |i\rangle) |0\rangle^{\otimes N_s}$. After O_g , it becomes

$$|\psi\rangle = \sum_{j=1}^{N_s} \sum_{g(i)=j} y_i |i\rangle |m_j\rangle. \quad (98)$$

Apply $R_z(s_j)$ in parallel to the N_s ancilla qubits, namely $\bigotimes_{j=1}^{N_s} \mathbf{R}_z(s_j)$. Since $\mathbf{R}_z(s) = \text{diag}(e^{-is/2}, e^{is/2})$ and only qubit j is $|1\rangle$ in $|m_j\rangle$, the phase accumulated by the ancilla register is

$$\exp \left[i \left(s_j - \frac{1}{2} \sum_{q=1}^{N_s} s_q \right) \right]. \quad (99)$$

For $N_s \neq 2$, choose

$$s_j = \hat{\theta}_j + \frac{\Theta}{2 - N_s}, \quad \Theta = \sum_{q=1}^{N_s} \hat{\theta}_q. \quad (100)$$

Equation (99) then equals $e^{i\hat{\theta}_j}$. For $N_s = 2$, after ignoring the global phase $e^{i(\hat{\theta}_1 + \hat{\theta}_2)/2}$, choose $s_j = \hat{\theta}_j - (\hat{\theta}_1 + \hat{\theta}_2)/2$. Finally, apply $O_g^\dagger$ to uncompute the ancillas, obtaining

$$O_g^\dagger \left(\sum_{j=1}^{N_s} \sum_{g(i)=j} y_i e^{i\hat{\theta}_j} |i\rangle |m_j\rangle \right) = (\mathbf{D}_\theta |\mathbf{y}\rangle) |0\rangle^{\otimes N_s}. \quad (101)$$

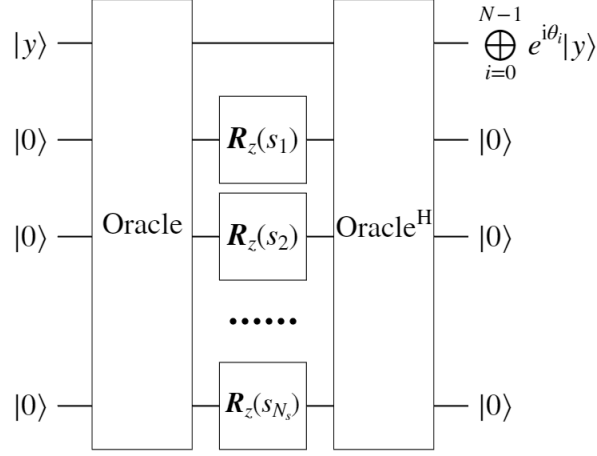


Figure 23 Circuit for a diagonal phase unitary using a one-hot lookup oracle

The corresponding circuit is shown in Fig. 23.

Figure 23 trades space for time. The parallel R_z layer has depth $O(1)$ but requires N_s ancilla qubits and $O(N_s)$ rotation gates. In the worst case, all N phases differ, so $N_s = N$. The oracle O_g can then use bucket-brigade quantum random-access memory, with $O(\log N)$ addressing depth [47]. Storage, control, and fault-tolerant implementation costs remain linear in N . Without structured $g(i)$ or available quantum random-access memory, generic oracle synthesis also generally requires resources growing with N . Thus, we assert only that a unitary implementation of Eq. (96) exists. Its actual complexity must be reassessed for a specific data-access model and hardware connectivity.

Appendix 3. Block-Hadamard test for the global loss

Let $\mathbf{K} = \sum_{i=0}^{M-1} a_i \mathbf{K}_i$. The quadratic form in the global loss is

$$\langle \mathbf{u} | \mathbf{K}^H \mathbf{K} | \mathbf{u} \rangle = \sum_{i=0}^{M-1} \sum_{j=0}^{M-1} a_i^* a_j \langle \mathbf{u} | \mathbf{K}_i^H \mathbf{K}_j | \mathbf{u} \rangle. \quad (102)$$

The loss requires the weighted sum of overlaps, not each individual value $\langle \mathbf{u} | \mathbf{K}_i^H \mathbf{K}_j | \mathbf{u} \rangle$. A separate Hadamard circuit for each term in Eq. (102) requires $O(M^2)$ circuit calls. We therefore partition the M unitary terms into M/B blocks of $B = 2^b$ terms, padding the term list with zero coefficients if B does not divide M . Define the weighted sum for block pair (k, l) as

$$c_{k,l} = \sum_{i=Bk}^{B(k+1)-1} \sum_{j=Bl}^{B(l+1)-1} a_i^* a_j \langle \mathbf{u} | \mathbf{K}_i^H \mathbf{K}_j | \mathbf{u} \rangle, \quad \langle \mathbf{u} | \mathbf{K}^H \mathbf{K} | \mathbf{u} \rangle = \sum_{k=0}^{M/B-1} \sum_{l=0}^{M/B-1} c_{k,l}. \quad (103)$$

The quantity $c_{k,l}$ is a weighted sum of B^2 overlaps and serves as a basic computational block for Eq. (102). If one circuit directly estimates $c_{k,l}$, the number of circuit configurations per loss evaluation falls from $O(M^2)$ to $O((M/B)^2)$. Define the normalized coefficient state of block k by

$$|\hat{\mathbf{a}}_k\rangle = \frac{1}{A_k} \sum_{i=Bk}^{B(k+1)-1} a_i |i - Bk\rangle = \mathbf{A}_k |0\rangle^{\otimes b}, \quad A_k = \sqrt{\sum_{i=Bk}^{B(k+1)-1} |a_i|^2}, \quad (104)$$

where $\mathbf{A}_k$ is a $B \times B$ unitary. Figure 24 shows the circuit estimating $c_{k,l}$ for $B = 2$, which places coefficient-state preparation and the corresponding SELECT operations for blocks k and l in the two branches of a Hadamard test.

The input at the left of the circuit is $|\psi\rangle = |0\rangle|0\rangle^{\otimes b}|0\rangle^{\otimes b}|0\rangle^{\otimes n}$. After the circuit layers, the final state is

Measure the first ancilla and denote the probabilities of $|0\rangle$ and $|1\rangle$ by ζ_0 and ζ_1 , respectively. They are

Therefore,

$$\text{Re}(c_{k,l}) = A_k A_l B(2\zeta_0 - 1) = A_k A_l B(1 - 2\zeta_1) = A_k A_l B(\zeta_0 - \zeta_1). \quad (107)$$

Similar circuits estimate all $c_{k,l}$ in batches. For real symmetric $\mathbf{K}$, summing all blocks gives the required real quadratic form. Blocking reduces the number of circuit configurations to $O((M/B)^2)$, or one configuration when $B = M$. Larger B reduces the number of configurations but increases circuit width and depth and lowers individual outcome probabilities.

References

- [1] Q. Gao, F. Wu, H. W. Zhang, W. X. Zhong, W. P. Howson, F. W. Williams, A fast precise integration method for structural dynamics problems, *Structural Engineering and Mechanics* 43 (2012) 1–13.
- [2] M. Mosby, K. Matous, Computational homogenization at extreme scales, *Extreme Mechanics Letters* 6 (2016) 68–74.
- [3] M. J. Turner, R. W. Clough, H. C. Martin, L. J. Topp, Stiffness and deflection analysis of complex structures, *Journal of the Aeronautical Sciences* 23 (1956) 805–823.
- [4] R. W. Clough, The finite element method in plane stress analysis, in: *Proceedings of the 2nd ASCE Conference on Electronic Computation*, Pittsburgh, PA (1960).
- [5] F. Wu, Y. Sun, W. X. Zhong, Inter-belt finite element for the analysis of incompressible material problems, *Applied Mathematics and Mechanics* 34 (2013) 1–9 (in Chinese).
- [6] W. Bangerth, C. Burstedde, T. Heister, M. Kronbichler, Algorithms and data structures for massively parallel generic adaptive finite element codes, *ACM Transactions on Mathematical Software* 38 (2011) Article 14, 1–28.
- [7] S. Koric, Q. Lu, E. Guleryuz, Evaluation of massively parallel linear sparse solvers on unstructured finite element meshes, *Computers & Structures* 141 (2014) 19–25.
- [8] F. Wu, Y. Zhao, Y. Yang, X. Zhang, N. Zhou, A new discrepancy for sample generation in stochastic response analyses of aerospace problems with uncertain parameters, *Chinese Journal of Aeronautics* 37 (2024) 192–211.
- [9] F. Wu, D. Huang, X. Xu, K. Zhao, N. Zhou, An adaptive divided-difference perturbation method for solving stochastic problems, *Structural Safety* 103 (2023) 102346.
- [10] R. P. Feynman, Simulating physics with computers, *International Journal of Theoretical Physics* 21 (1982) 467–488.
- [11] S. Lloyd, Universal quantum simulators, *Science* 273 (1996) 1073–1078.
- [12] Y. Kim, A. Eddins, S. Anand, et al., Evidence for the utility of quantum computing before fault tolerance, *Nature* 618 (2023) 500–505.
- [13] J. Preskill, Quantum computing in the NISQ era and beyond, *Quantum* 2 (2018) 79.

- [14] A. Peruzzo, J. McClean, P. Shadbolt, et al., A variational eigenvalue solver on a photonic quantum processor, *Nature Communications* 5 (2014) 4213.
- [15] Y. Cao, J. Romero, J. P. Olson, et al., Quantum chemistry in the age of quantum computing, *Chemical Reviews* 119 (2019) 10856–10915.
- [16] S. McArdle, S. Endo, A. Aspuru-Guzik, S. C. Benjamin, X. Yuan, Quantum computational chemistry, *Reviews of Modern Physics* 92 (2020) 015003.
- [17] R. Au-Yeung, B. Camino, O. Rathore, V. Kendon, Quantum algorithms for scientific computing, *Reports on Progress in Physics* 87 (2024) 116001.
- [18] G. Tosti Balducci, B. Chen, M. Möller, M. Gerritsma, R. De Breuker, Review and perspectives in quantum computing for partial differential equations in structural mechanics, *Frontiers in Mechanical Engineering* 8 (2022) 914241.
- [19] Z. Meng, Y. Yang, Quantum computing of fluid dynamics using the hydrodynamic Schrödinger equation, *Physical Review Research* 5 (2023) 033182.
- [20] Z. Meng, Y. Yang, Quantum spin representation for the Navier–Stokes equation, *Physical Review Research* 6 (2024) 043130.
- [21] Z. Meng, J. Zhong, S. Xu, et al., Simulating unsteady flows on a superconducting quantum processor, *Communications Physics* 7 (2024) 349.
- [22] F. Wu, Y. Yang, L. Zhu, C. Li, Y. Guo, X. Guo, A voxel-based quantum computing method (VBQC) for solid mechanics problem, *arXiv:2606.03515* (2026).
- [23] H.-W. Zhang, J.-K. Wu, J. Lü, Z.-D. Fu, Extended multiscale finite element method for mechanical analysis of heterogeneous materials, *Acta Mechanica Sinica* 26 (2010) 899–920.
- [24] Y. Xu, H. Hu, Potential energy minimization for structural analysis via decomposition-free quantum computing, *Computers & Structures* 330 (2026) 108309.
- [25] S. Jin, N. Liu, Y. Yu, Quantum simulation of partial differential equations via Schrödingerization, *Physical Review Letters* 133 (2024) 230602.
- [26] Y. Xu, K. Zhao, H.-T. Liu, K. Huang, Z. Liu, H. Fan, H. Hu, Hamiltonian simulation of elastodynamics on a quantum computer via energy conservation mapping, *Journal of the Mechanics and Physics of Solids* 213 (2026) 106639.
- [27] Y. Xu, H. Hu, Decomposition-free variational quantum linear solver: Application in computational mechanics, *Computer Methods in Applied Mechanics and Engineering* 447 (2025) 118396.
- [28] O. M. Raisuddin, S. De, FEqa: Finite element computations on quantum annealers, *Computer Methods in Applied Mechanics and Engineering* 395 (2022) 115014.
- [29] A. W. Harrow, A. Hassidim, S. Lloyd, Quantum algorithm for linear systems of equations, *Physical Review Letters* 103 (2009) 150502.
- [30] A. Montanaro, S. Pallister, Quantum algorithms and the finite element method, *Physical Review A* 93 (2016) 032324.
- [31] M. Cerezo, A. Arrasmith, R. Babbush, et al., Variational quantum algorithms, *Nature Reviews Physics* 3 (2021) 625–644.
- [32] C. Bravo-Prieto, R. LaRose, M. Cerezo, Y. Subasi, L. Cincio, P. J. Coles, Variational quantum linear solver, *Quantum* 7 (2023) 1188.
- [33] J.-W. Ying, J.-C. Shen, L. Zhou, W. Zhong, M.-M. Du, Y.-B. Sheng, Preparing a fast Pauli decomposition for variational quantum solving linear equations, *Annalen der Physik* 535 (2023) 2300212.
- [34] Y. Sato, R. Kondo, S. Koide, H. Takamatsu, N. Imoto, Variational quantum algorithm based on the minimum potential energy for solving the Poisson equation, *Physical Review A* 104 (2021) 052409.
- [35] H. Patil, Y. Wang, P. S. Krstić, Variational quantum linear solver with a dynamic ansatz, *Physical Review A* 105 (2022) 012423.
- [36] A. Pellow-Jarman, I. Sinayskiy, A. Pillay, F. Petruccione, A comparison of various classical optimizers for a variational quantum linear solver, *Quantum Information Processing* 20 (2021) 202.
- [37] C. J. Trahan, M. Loveland, N. Davis, E. Ellison, A variational quantum linear solver application to discrete finite-element methods, *Entropy* 25 (2023) 580.
- [38] M. Ali, M. Kabel, Performance study of variational quantum algorithms for solving the Poisson equation on a quantum computer, *Physical Review Applied* 20 (2023) 014054.
- [39] A. Arora, B. M. Ward, C. Oskay, An implementation of the finite element method in hybrid classical/quantum computers, *Finite Elements in Analysis and Design* 248 (2025) 104354.
- [40] Y. Liu, J. Liu, J. R. Raney, P. Wang, Quantum computing for solid mechanics and structural engineering—A demonstration with variational quantum eigensolver, *Extreme Mechanics Letters* 67 (2024) 102117.
- [41] J. R. McClean, S. Boixo, V. N. Smelyanskiy, R. Babbush, H. Neven, Barren plateaus in quantum neural network training landscapes, *Nature Communications* 9 (2018) 4812.
- [42] M. Cerezo, A. Sone, T. Volkoff, L. Cincio, P. J. Coles, Cost function dependent barren plateaus in shallow parametrized quantum circuits, *Nature Communications* 12 (2021) 1791.
- [43] A. M. Childs, N. Wiebe, Hamiltonian simulation using linear combinations of unitary operations, *Quantum Information and Computation* 12 (2012) 901–924.
- [44] F. Wu, C. Li, Y.-X. Yang, L. Zhu, X. Guo, Quantum simulation of Hamiltonian in solid mechanics based on voxel representation, *Chinese Journal of Computational Mechanics* 42 (2025) 329–338 (in Chinese).
- [45] S. Chakraborty, Implementing any linear combination of unitaries on intermediate-term quantum computers, *Quantum* 8 (2024) 1496.
- [46] M. Möttönen, J. J. Vartiainen, V. Bergholm, M. M. Salomaa, Transformation of quantum states using uniformly controlled rotations, *Quantum Information and Computation* 5 (2005) 467–473.
- [47] V. Giovannetti, S. Lloyd, L. Maccone, Architectures for a quantum random access memory, *Physical Review A* 78 (2008) 052310.
- [48] H.-Y. Huang, K. Bharti, P. Rebentrost, Near-term quantum algorithms for linear systems of equations with regression loss functions, *New Journal of Physics* 23 (2021) 113021.
- [49] M. Schneider, Voxel-based finite elements with hourglass control in fast Fourier transform-based computational homogenization, *International Journal for Numerical Methods in Engineering* 123 (2022) 6286–6313.
- [50] L. Hantzko, L. Binkowski, S. Gupta, Tensorized Pauli decomposition algorithm, *Physica Scripta* 99 (2024) 085128.
- [51] F. Wu, C. Li, X. Wu, Y. Guo, X. Guo, A matrix sparsity-based Pauli decomposition algorithm, *Acta Mechanica Sinica* (2026) in press.
- [52] V. V. Shende, S. S. Bullock, I. L. Markov, Synthesis of quantum-logic circuits, *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 25 (2006) 1000–1010.
- [53] A. V. Soudackov, D. G. A. Cabral, B. C. Allen, et al., QFlux: Quantum circuit implementations of molecular dynamics. Part III—State initialization and unitary decomposition, *ChemRxiv* (2026).